REȚELE DE CALCULATOARE · SEMESTRUL 1 · SĂPTĂMÂNA 2

Arhitectură și performanță în scheme

Niveluri, modele de referință și ce este în spatele întârzierii — curs, seminar și laboratoare

Conf.univ.dr. Popescu-Bodorin Nicolaie, 2026
+40722882471, bodorin(on)live.com, https://lmrec.org/bodorin/

Convenție de terminologie

La prima apariție în fiecare parte a documentului, termenii de specialitate traduși în română sunt urmați, în paranteză, de termenul consacrat în limba engleză sau de forma „romgleză” folosită curent de practicieni — de exemplu, **comutator de pachete** (*packet switch*), **memorie tampon** (*buffer*, în romgleză „buffer”). Comenzile, numele câmpurilor din Wireshark și rezultatele afișate de programe sunt lăsate în engleză, așa cum apar pe ecran.

termen românesc	termen englezesc	formă romgleză
comutator de pachete	(packet switch)	„buffer”, „bufferbloat”

rămân în engleză, așa cum apar pe ecran:

```
Time to Live: 64 · tcp.flags.syn == 1 · Destination Port: 80 · pktmon start --capture
```

Ideea-cheie Termenul românesc pentru înțelegere, cel englezesc pentru documentație și pentru motoarele de căutare.

Cum este organizată săptămâna

A · Cursul 2
Arhitectură și performanță

B · Seminarul 2
Antete, cozi, debit

C · Laboratorul 2 · Linux
Wireshark și tcpdump

D · Laboratorul 2 · Windows
Wireshark și pktmon

Concepte

niveluri, încapsulare, OSI (Open Systems Interconnection) și TCP/IP (Transmission Control Protocol / Internet Protocol), standarde, cozi, debit, capăt-la-capăt

Calcule

decodificarea unui cadru, cost suplimentar, intensitatea traficului, legea lui Little, legături înguste, BDP (bandwidth–delay product, produsul lățime de bandă–întârziere)

Pachete pe Linux

Wireshark și tcpdump: o cerere web deschisă nivel cu nivel

Pachete pe Windows

Wireshark, Npcap (driverul de captură de pachete al proiectului Nmap) și pktmon, inclus în sistem

Ideea-cheie Citiți prezentarea de la stânga la dreapta: laboratoarele deschid, în pachete reale, antetele desenate la curs.

Partea A · Cursul 2: arhitectură și performanță

A · Cursul 2
Arhitectură și performanță

→

B · Seminarul 2
Antete, cozi, debit

→

C · Laboratorul 2 · Linux
Wireshark și tcpdump

→

D · Laboratorul 2 · Windows
Wireshark și pktmon

În această parte

1 · Două întrebări și o enigmă

2 · De ce niveluri: serviciu, interfață, protocol

3 · Încapsulare și demultiplexare

4 · Modele de referință: OSI, TCP/IP, talia îngustă

5 · Cine face regulile: standarde și RFC-uri

6 · Întârzierea în coadă și pierderile

7 · Debitul și produsul lățime de bandă–întârziere

8 · Argumentul capăt-la-capăt

9 · Cum măsoară ping și traceroute întârzierea

Rezumat · întrebări · exerciții

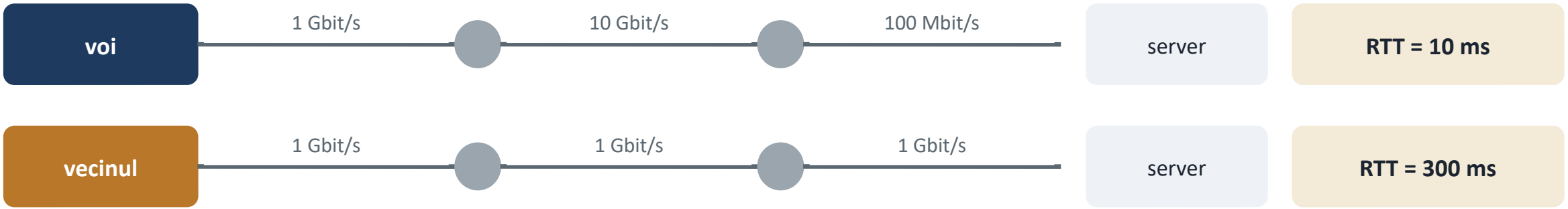
TCP (Transmission Control Protocol) · OSI (Open Systems Interconnection) · RFC (Request for Comments) · IP (Internet Protocol)

1 · Două întrebări și o enigmă de ținut minte

Cum este împărțită munca?
de ce niveluri, care niveluri, ce trece între ele → secțiunile 2–5

Cât de bine funcționează?
întârziere, pierderi, debit — și de ce o încărcare aproape de 100 % strică o rețea → secțiunile 6–9

Întrebarea de început: cine termină primul?



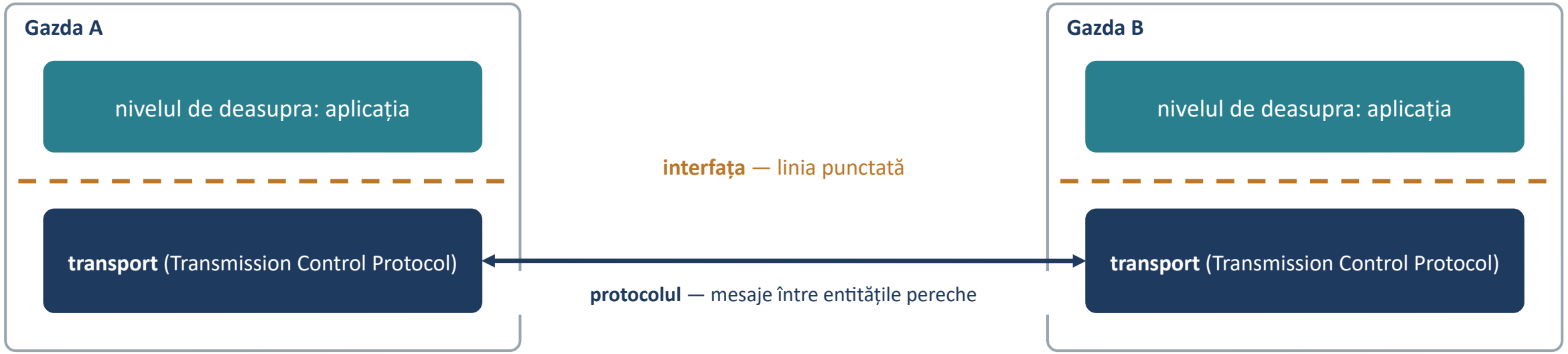
o descărcare de **1 MB** — cine termină primul? ?

o descărcare de **10 GB** — cine termină primul? ?

Ideea-cheie Păstrați-vă presupunerile: răspunsul vine în secțiunea 7, după întârziere și debit.

RTT (round-trip time, timp dus-întors)

2 · Serviciu, interfață, protocol — trei cuvinte de ținut separat



1 Serviciu

Ce oferă un nivel nivelului de deasupra.

de ex. livrare fiabilă și în ordine a unui flux de octeți între două procese

2 Interfață

Cum îl cere nivelul de deasupra: operații și parametri.

de ex. apelurile socket `connect`, `send`, `recv`, `close`

3 Protocol

Cum îl realizează nivelul, prin mesaje cu entitatea pereche.

de ex. segmente TCP: numere de secvență, ACK-uri (acknowledgements, confirmări), retransmisii

Ideea-cheie Serviciu = ce se oferă, interfață = cum îl ceri, protocol = cum se realizează împreună cu entitatea pereche.

ACK (acknowledgement, confirmare)

2 · O interfață fixă permite schimbarea protocolului de dedesubt

navigatorul web (browser) — scris o dată, neschimbat

interfața socket: connect · send · recv · close — fixă de decenii

TCP (Transmission Control Protocol) dedesubt — algoritmul de control al congestiei, înlocuit de-a lungul anilor:

Reno
sfârșitul anilor 1980 – anii 1990

→

CUBIC
implicit în Linux din anii 2000

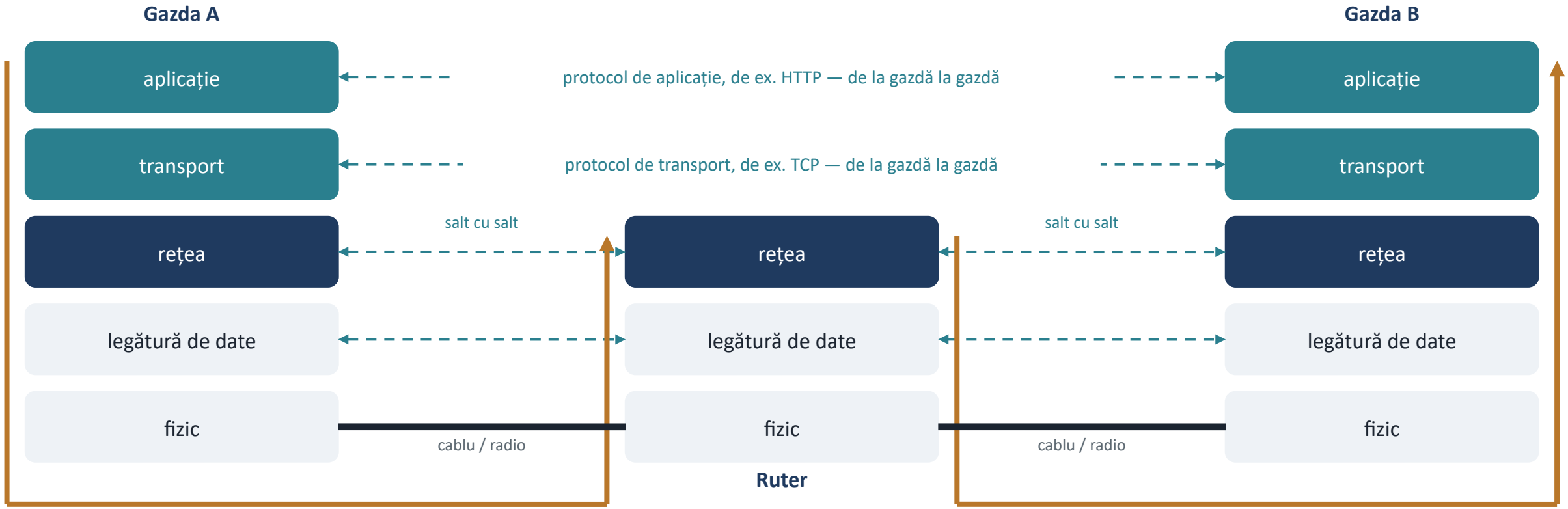
→

BBR
2016 → (Bottleneck Bandwidth and Round-trip propagation time)

1 **Serviciul** (flux fiabil de octeți) și **interfața** au rămas; s-a schimbat doar **protocolul** din interior.

Ideea-cheie Separarea serviciului și a interfeței de protocol face posibilă schimbarea fără a strica aplicațiile.

2.1 · Entități pereche: logic pe orizontală, real jos–peste–sus



1 **Linie punctată, turcoaz**: conversația logică dintre entitățile pereche.

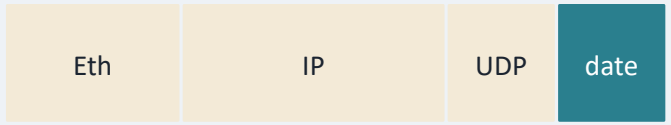
2 **Linie continuă, chihlimbarie**: drumul real al datelor.

Ideea-cheie Ruterele nu participă la protocoalele de aplicație și de transport: acestea rulează doar între cele două gazde.

HTTP (HyperText Transfer Protocol) · TCP (Transmission Control Protocol)

2.2 · Prețul împărțirii pe niveluri

1 Cost suplimentar (overhead)



un mesaj de 20 de octeți într-un cadru de 86 de octeți: **23 %** util

Fiecare nivel adaugă un antet — mesajele mici plătesc cel mai mult.

2 Funcții dublate

transport: sumă de control ✓

legătură: CRC (cyclic redundancy check, control ciclic de redundanță) ✓

uneori sunt necesare ambele, alteori nu (§ 8)

Detecția erorilor la nivelul legăturii **și** la transport.

3 Informație ascunsă

TCP: „s-a pierdut un pachet”

granița dintre niveluri

legătura știe: congestie?
sau interferență radio?

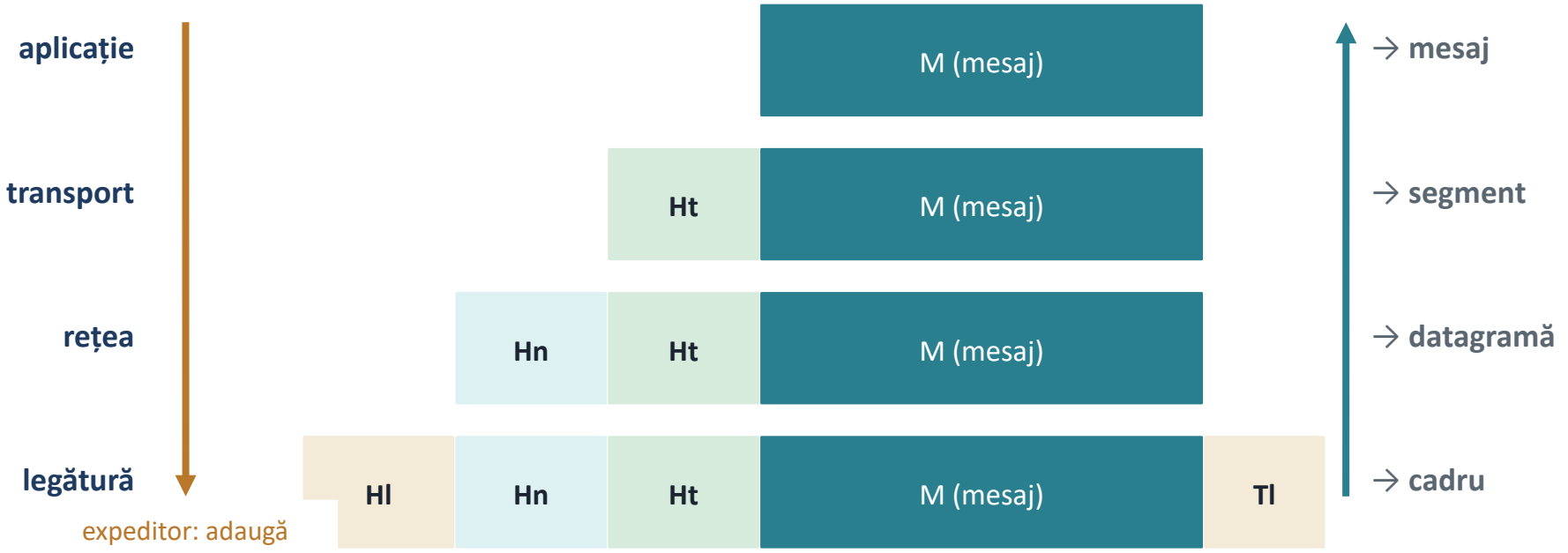
TCP vede o pierdere, dar nu poate vedea **de ce**.

Proiectarea inter-niveluri (*cross-layer design*) — o excepție deliberată: de ex. o aplicație video pe telefon care se adaptează la condițiile radio despre care este informată.

Ideea-cheie Nivelurile costă octeți de antet, muncă repetată și informație ascunsă — un preț care de obicei merită plătit.

TCP (Transmission Control Protocol) · UDP (User Datagram Protocol) · IP (Internet Protocol)

3 · Încapsularea: fiecare nivel împachetează ce primește



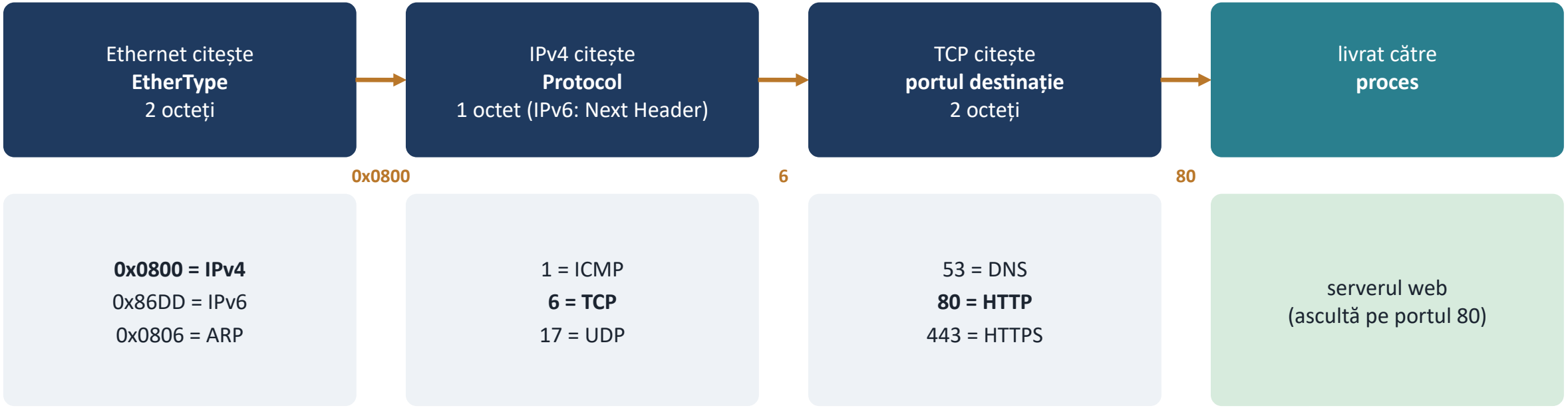
Ht / Hn / Hl = antetul (header) de transport / rețea / legătură · Tl = sfârșitul de cadru (trailer)

1 Unitatea primită de sus = **sarcina utilă** (*payload*), în limbajul OSI (Open Systems Interconnection) o **SDU (service data unit, unitate de date a serviciului)**.

2 Sarcina utilă + antetul acestui nivel = **PDU (protocol data unit, unitate de date a protocolului)** a nivelului: segment, datagramă, cadru.

Ideea-cheie La coborâre, fiecare nivel își adaugă antetul; la urcare, fiecare nivel și-l îndepărtează pe al său și dă restul mai departe.

3 · Demultiplexarea: un câmp de antet alege următorul destinatar



1 **EtherType → Protocol → port**: câte un câmp pe nivel alege destinatarul de deasupra.

2 Îngroșat = exemplul de față: un cadru care poartă IPv4, care poartă TCP, pentru portul 80.

Ideea-cheie Trei câmpuri livrează un cadru procesului potrivit: EtherType, Protocol din IP, portul destinație.

HTTPS (HyperText Transfer Protocol Secure) · HTTP (HyperText Transfer Protocol) · ICMP (Internet Control Message Protocol) · IPv4 (Internet Protocol version 4) · IPv6 (Internet Protocol version 6) · TCP (Transmission Control Protocol) · UDP (User Datagram Protocol) · DNS (Domain Name System) · ARP (Address Resolution Protocol) · IP (Internet Protocol)

3 · Numerele de port: trei intervale, un registru

0 65535

bine-cunoscute 0 – 1023	**înregistrate** 1024 – 49151	**dinamice** 49152 – 65535
servicii standard: 53 DNS · 80 HTTP · 443 HTTPS	anumite aplicații, de ex. 8080 (HTTP alternativ)	alese de sistemul de operare pentru clienți, de ex. 51234

IANA
Internet Assigned Numbers Authority
ține registrele publice: numere de port, numere de protocol IP, …

IEEE
Institute of Electrical and Electronics Engineers
atribuie valorile EtherType (nivelul legătură de date)

Ideea-cheie Serverele ascultă pe porturi bine-cunoscute; sistemul de operare dă fiecărui socket client unul dinamic.

HTTPS (HyperText Transfer Protocol Secure) · HTTP (HyperText Transfer Protocol) · DNS (Domain Name System) · IP (Internet Protocol)

3.1 · O cerere web de 400 de octeți devine un cadru de 458

antet Ethernet 14 B · legătură	**antet IPv4** 20 B · rețea	**antet TCP** 20 (+ opțiuni) B · transport	**cererea HTTP** 400 B · aplicație	**sfârșit de cadru** 4 B · legătură
MAC destinație · MAC sursă · EtherType 0x0800	IP sursă și destinație · TTL · Protocol = 6 · suma de control a antetului	port sursă 51234 · port destinație 80 · numere de secvență și ACK	`GET /index.html HTTP/1.1` + liniile de antet	FCS, CRC-32

la scară:



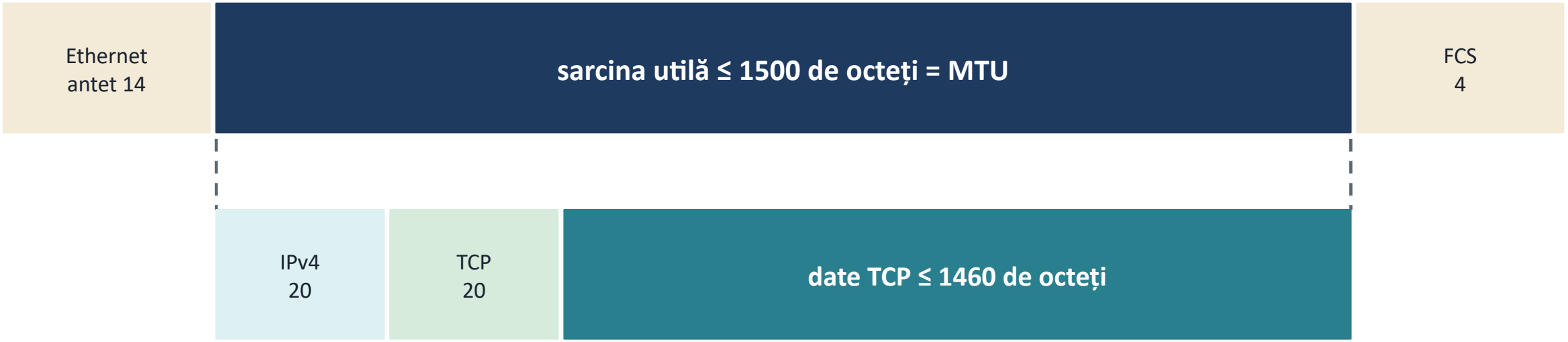
58 de octeți de antete + 400 de octeți HTTP = 458 de octeți; 58 / 458 ≈ 13 % cost suplimentar

MAC = Media Access Control · TTL = time to live, timp de viață · FCS = frame check sequence, secvența de verificare a cadrului · CRC = cyclic redundancy check, control ciclic de redundanță

Ideea-cheie O cerere, trei niveluri de împachetare: 14 + 20 + 20 de octeți în față, 4 octeți în spate.

HTTP (HyperText Transfer Protocol) · IPv4 (Internet Protocol version 4) · TCP (Transmission Control Protocol) · ACK (acknowledgement, confirmare) · IP (Internet Protocol)

3.1 · MTU: de ce apare 1460 peste tot



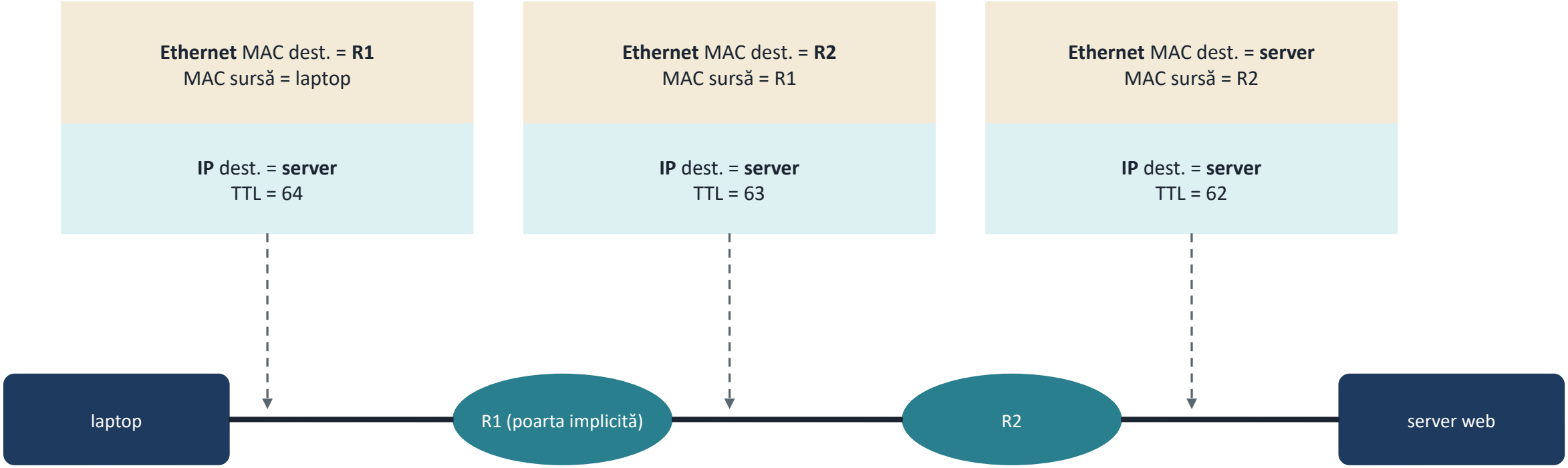
peste IPv4: 1500 − 20 − 20 = **1460**
MSS (maximum segment size, dimensiunea maximă a segmentului) uzuală

peste IPv6: 1500 − 40 − 20 = **1440**
antetul IPv6 are 40 de octeți

Ideea-cheie MTU 1500 minus antetele IP și TCP: cel mult 1460 de octeți de date pe cadru Ethernet, peste IPv4.

IPv4 (Internet Protocol version 4) · IPv6 (Internet Protocol version 6) · TCP (Transmission Control Protocol) · MTU (maximum transmission unit, unitatea maximă de transmisie) · FCS (frame check sequence, secvența de verificare a cadrului) · IP (Internet Protocol)

3.1 · Spre ce indică adresa MAC destinație



1 Adresele **MAC** (Media Access Control): următorul salt pe această legătură — rescrise la fiecare ruter.

2 **Antetul IP**: capăt-la-capăt; doar **TTL** (time to live, timp de viață) scade cu unu la fiecare ruter.

Ideea-cheie Adresa IP = destinația finală; adresa MAC = următorul salt. Veți verifica singuri acest lucru în Laboratorul 2.

IP (Internet Protocol)

4.1 · Modelul de referință OSI: șapte niveluri (ISO 7498, 1984)

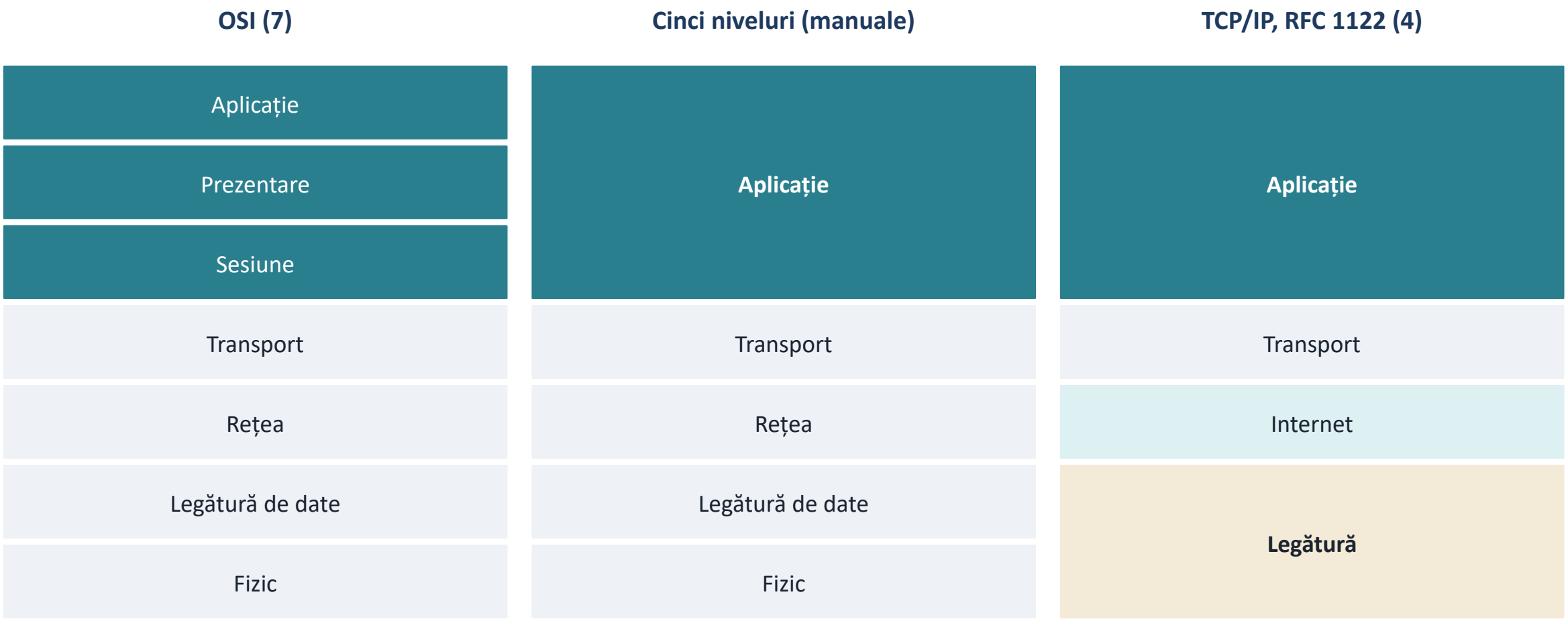
#	Nivelul OSI	Funcția	Unde se regăsește în Internet
7	**Aplicație**	servicii folosite direct de aplicații	HTTP, SMTP, DNS
6	**Prezentare**	reprezentare: codificare, compresie, criptare	în aplicații și biblioteci: UTF-8, JSON, TLS
5	**Sesiune**	controlul dialogului, sincronizare, puncte de control	în aplicații: sesiuni HTTP, reluarea TLS
4	**Transport**	livrare capăt-la-capăt între procese	TCP, UDP, QUIC
3	**Rețea**	dirijare și retransmitere între rețele	IP
2	**Legătură de date**	cadre între vecini; detecția erorilor	Ethernet, Wi-Fi
1	**Fizic**	biți bruți pe un mediu	semnale pe cupru, fibră, radio

Rândurile nisipii: niveluri fără protocol separat în Internet — munca lor se face în interiorul aplicațiilor.

Ideea-cheie Mnemonic englezesc, de jos în sus: „Please Do Not Throw Sausage Pizza Away” (Physical … Application).

UTF-8 (Unicode Transformation Format, 8-bit) · HTTP (HyperText Transfer Protocol) · QUIC (protocol de transport peste UDP, User Datagram Protocol; numele inițial: Quick UDP Internet Connections) · SMTP (Simple Mail Transfer Protocol) · JSON (JavaScript Object Notation) · TCP (Transmission Control Protocol) · UDP (User Datagram Protocol) · TLS (Transport Layer Security) · DNS (Domain Name System) · OSI (Open Systems Interconnection) · ISO (International Organization for Standardization) · IP (Internet Protocol)

4.2 · Trei modele alăturate



Ideea-cheie Internetul include prezentarea și sesiunea în aplicații, iar legătura de date și nivelul fizic într-un singur nivel legătură.

TCP (Transmission Control Protocol) · OSI (Open Systems Interconnection) · RFC (Request for Comments) · IP (Internet Protocol)

4.2 · De ce a câștigat TCP/IP — și ce a lăsat în urmă OSI

TCP/IP	protocoale care rulau pe mașini reale	→ distribuite gratuit cu Berkeley UNIX, începutul anilor 1980	→ **rețeaua lumii**
OSI	model făcut de comitete, sfârșitul anilor 1970	→ protocoale târzii, complexe, scumpe	→ **protocoalele s-au stins**

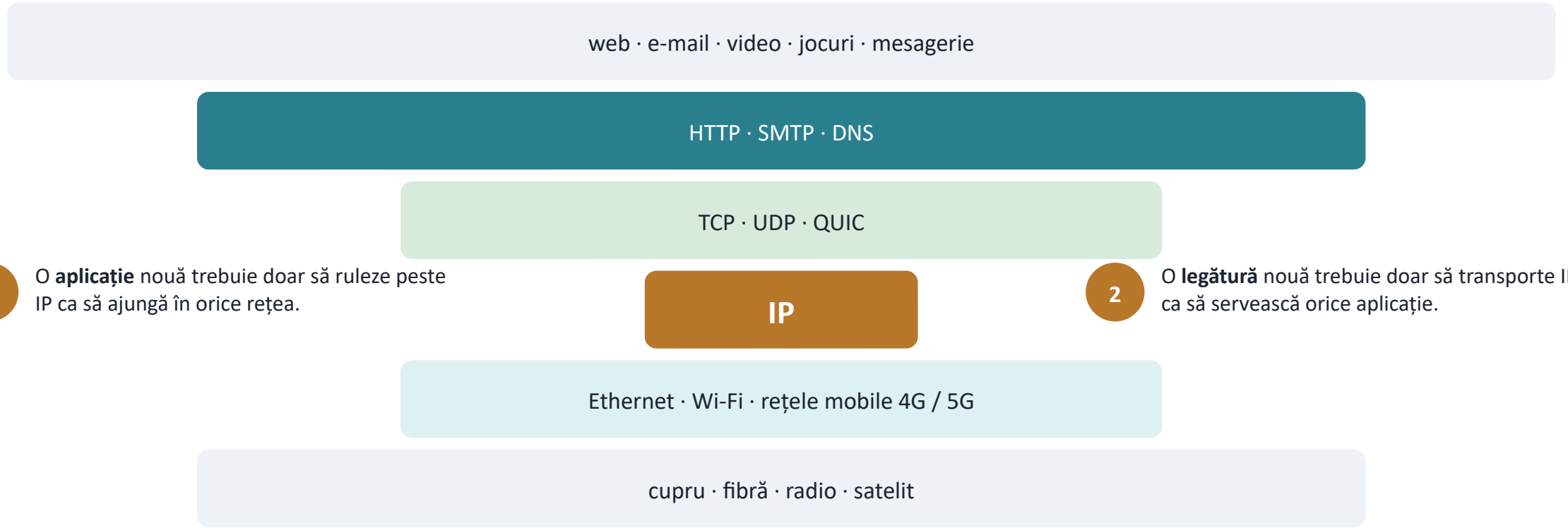
Ce a supraviețuit din OSI: vocabularul

„un comutator de nivel 2 (layer-2 switch)”	„un ruter de nivel 3 (layer-3 router)”	„un echilibrator de încărcare de nivel 7 (layer-7 load balancer)”

Ideea-cheie TCP/IP a câștigat cu cod care rula; OSI supraviețuiește prin cuvintele cu care numerotăm nivelurile.

TCP (Transmission Control Protocol) · OSI (Open Systems Interconnection) · IP (Internet Protocol)

4.3 · Talia îngustă: o clepsidră cu IP la mijloc



1 O **aplicație** nouă trebuie doar să ruleze peste IP ca să ajungă în orice rețea.

2 O **legătură** nouă trebuie doar să transporte IP ca să servească orice aplicație.

Costul: talia este foarte greu de schimbat — trecerea de la IPv4 la IPv6 durează de peste douăzeci și cinci de ani.

Ideea-cheie Totul rulează peste IP și IP rulează peste orice: ușor de extins la margini, greu de schimbat la mijloc.

HTTP (HyperText Transfer Protocol) · QUIC (protocol de transport peste UDP, User Datagram Protocol; numele inițial: Quick UDP Internet Connections) · SMTP (Simple Mail Transfer Protocol) · IPv4 (Internet Protocol version 4) · IPv6 (Internet Protocol version 6) · TCP (Transmission Control Protocol) · UDP (User Datagram Protocol) · DNS (Domain Name System) · IP (Internet Protocol) · 4G (a patra generație) · 5G (a cincea generație)

5 · Cine face regulile

protocoale	**IETF** Internet Engineering Task Force	IP, TCP, UDP, QUIC, DNS, HTTP, TLS … deschis oricui; lucrul se face public, în liste de discuții și grupuri de lucru
legături locale	comitetele **IEEE 802** Institute of Electrical and Electronics Engineers	802.3 Ethernet · 802.11 Wi-Fi · 802.1 punți și LAN-uri virtuale (rețele locale virtuale) — rețele locale și metropolitane
telecom	**ITU-T** și **3GPP** International Telecommunication Union – Telecommunication Standardization Sector · 3rd Generation Partnership Project	standarde de telecomunicații; 3GPP specifică sistemele mobile 4G (a patra generație) și 5G (a cincea generație)
Web-ul	**W3C** și **WHATWG** World Wide Web Consortium · Web Hypertext Application Technology Working Group	HTML (HyperText Markup Language), CSS (Cascading Style Sheets), API-urile navigatoarelor (application programming interfaces, interfețe de programare)
numere	**IANA** și cele cinci **RIR-uri** Internet Assigned Numbers Authority · Regional Internet Registry, registru regional de Internet (câte unul pe regiune)	IANA dă blocuri de adrese RIR-urilor; România → **RIPE NCC** (Réseaux IP Européens Network Coordination Centre)

Ideea-cheie IETF scrie protocoalele Internetului, IEEE legăturile lui locale, iar IANA și RIR-urile îi împart numerele.

HTTP (HyperText Transfer Protocol) · QUIC (protocol de transport peste UDP, User Datagram Protocol; numele inițial: Quick UDP Internet Connections) · TCP (Transmission Control Protocol) · UDP (User Datagram Protocol) · TLS (Transport Layer Security) · DNS (Domain Name System) · LAN (local area network, rețea locală) · API (application programming interface, interfață de programare) · IP (Internet Protocol)

5 · De la Internet-Draft la Internet Standard

Internet-Draft
îl poate scrie oricine

→

grupul de lucru
îl adoptă, ajunge la consens aproximativ

→

RFC: Proposed Standard
publicat, nu mai este modificat niciodată

→

Internet Standard
matur, implementat pe scară largă

cele două niveluri ale traseului de standardizare de la RFC 6410

RFC-urile nu se modifică — se înlocuiesc

RFC 7230–7235
2014

îl înlocuiește ←

RFC 2616
HTTP/1.1, 1999

Nu orice RFC este un standard

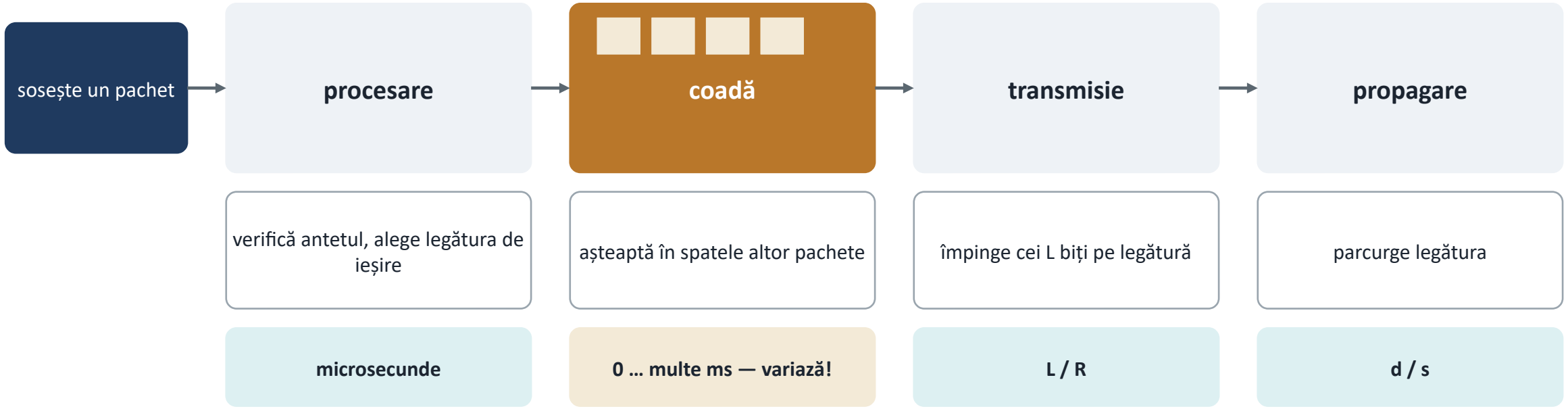
- informativ
- experimental
- istoric
- 1 aprilie: glume

„Respingem regii, președinții și votul. Credem în consensul aproximativ și în codul care rulează.”
— David Clark, 1992 (*rough consensus and running code*)

Ideea-cheie O specificație este luată în serios atunci când există implementări independente care funcționează împreună.

HTTP (HyperText Transfer Protocol) · RFC (Request for Comments)

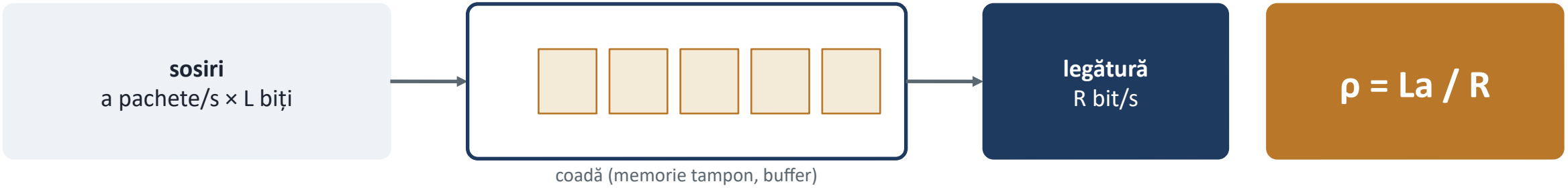
6 · Patru întârzieri la un ruter — doar una este imprevizibilă



1 L = lungimea pachetului (biți) · R = rata legăturii (bit/s) · d = lungimea legăturii · s = viteza de propagare. Așteptarea în coadă este cea interesantă: depinde de traficul **altora**.

Ideea-cheie Trei întârzieri sunt formule; întârzierea în coadă este statistică — subiectul acestei secțiuni.

6.1 · Intensitatea traficului ρ = La / R



ρ > 1

biții sosesc mai repede decât pleacă: coada crește **fără limită**; memoriile tampon sunt finite → **pierderi**. Nicio dimensiune de buffer nu rezolvă asta.

ρ ≤ 1, sosiri regulate

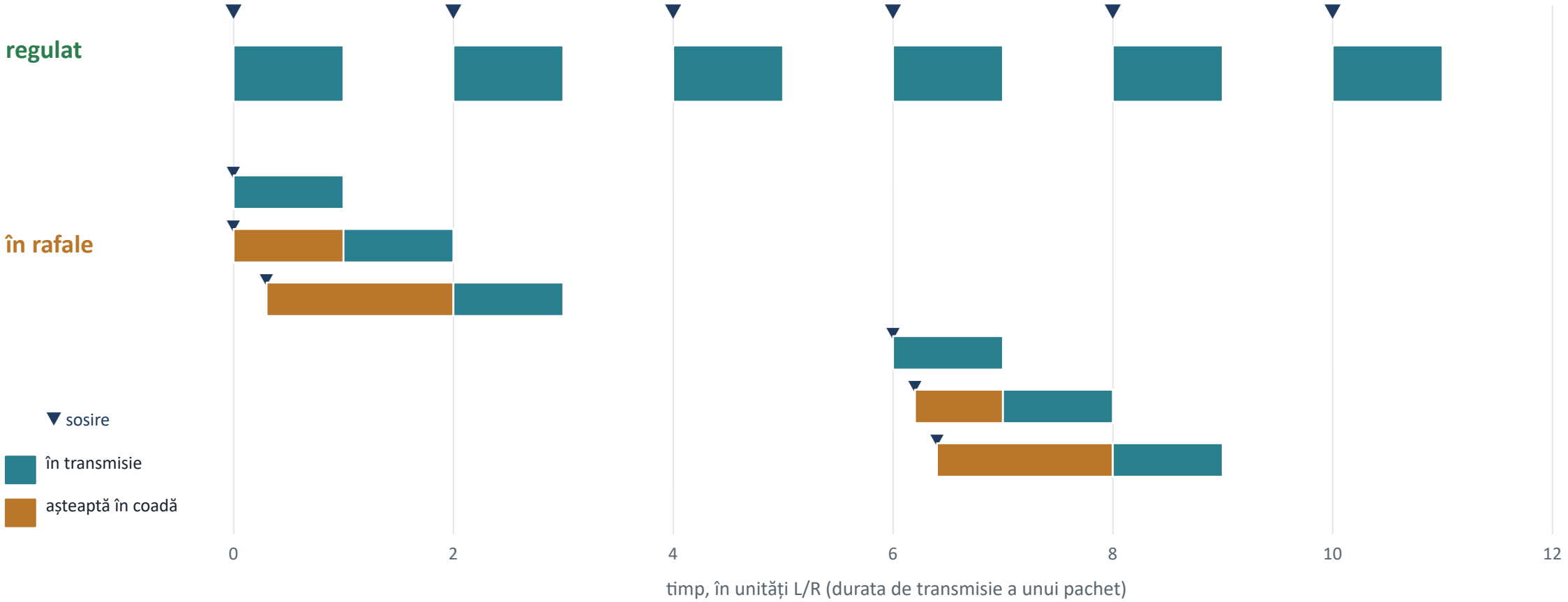
câte un pachet la fiecare L/R secunde: **niciun pachet nu așteaptă**.

ρ ≤ 1, sosiri aleatoare, în rafale

cazul real: pachetele sosesc grupat și **așteaptă**; cu cât ρ e mai aproape de 1, cu atât mai mult și mai des.

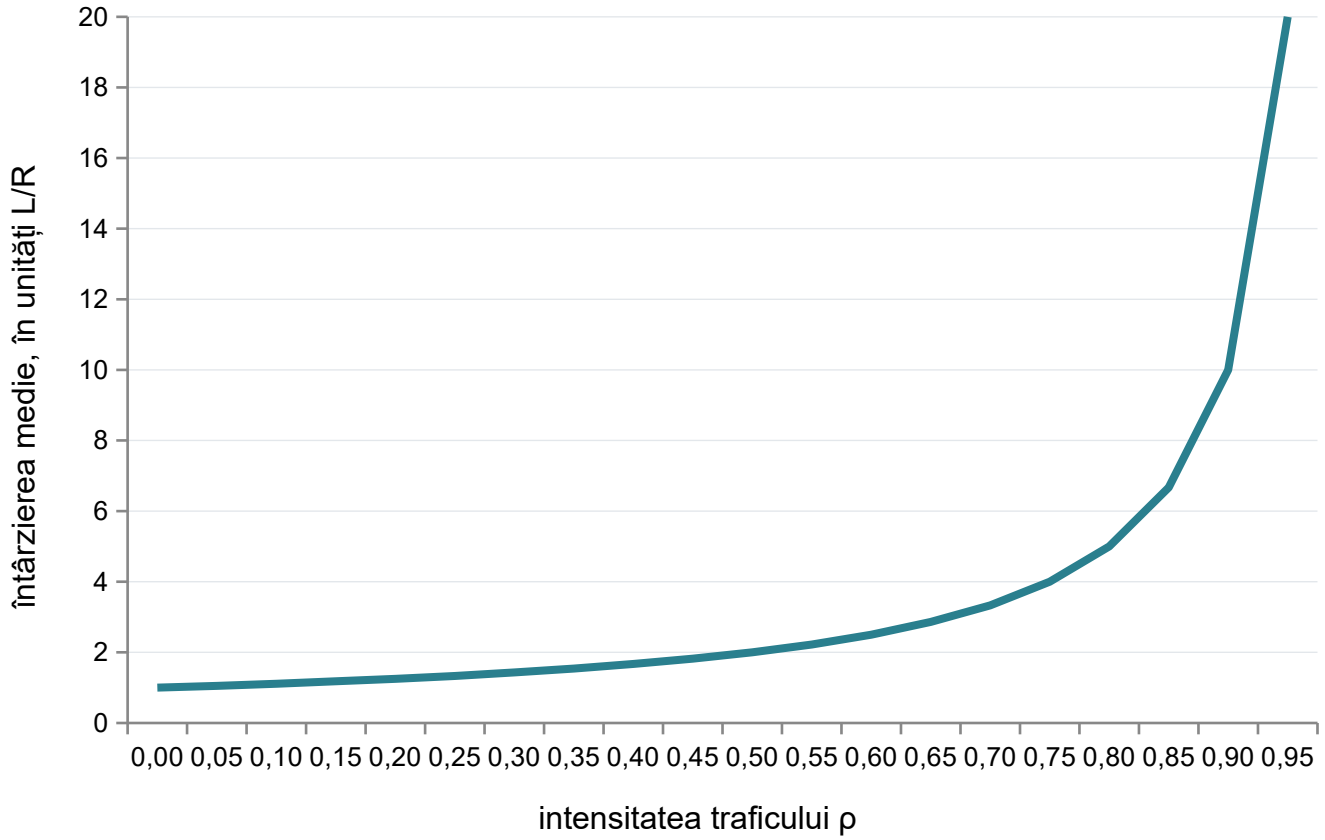
Ideea-cheie Prima regulă a ingineriei traficului: proiectați astfel încât ρ să rămână sub 1.

6.1 · Aceeași încărcare, așteptări foarte diferite



Ideea-cheie Cozile vin din rafale: cu sosiri aleatoare, așteptările apar cu mult înainte ca legătura să fie plină.

6.2 · Cât de repede crește întârzierea: modelul M/M/1



$$d = (L/R) / (1 - \rho)$$

încărcarea ρ	întârzierea
50 %	2 × L/R
80 %	5 × L/R
90 %	10 × L/R
95 %	20 × L/R

1 90 % → 95 %: **+5 % trafic dublează întârzierea**. Operatorii măresc capacitatea cu mult înainte de „plin”.

Ideea-cheie Traficul real nu este M/M/1 — credeți forma, nu cifrele: întârzierea explodează când ρ se apropie de 1.

M/M/1 (sosiri markoviene / durate de servire markoviene / 1 server)

6.3 · Pierderi, dimensiunea memoriei tampon și bufferbloat

Aruncare la coadă plină (tail drop)



memorie tampon: normală, plină

Coadă plină → pachetele care sosesc sunt **aruncate**. TCP le retransmite (întârziere în plus); fără TCP pur și simplu lipsesc (sacadări).

Bufferbloat (≈ 2010)



memorie tampon: foarte mare, plină

Buffer uriaș și plin = o coadă lungă. O singură încărcare mare adaugă **sute de ms** fiecărui pachet — inclusiv celor ale unui apel video.

Gestionarea activă a cozii



memorie tampon: normală, aproape goală

Algoritmi precum **CoDel** (Controlled Delay) aruncă sau marchează pachetele **devreme**, menținând scurtă coada permanentă.

Ideea-cheie Un buffer mare și plin este doar o coadă lungă: remediul este gestionarea cozii, nu doar dimensionarea ei. (Laborator în semestrul 2.)

TCP (Transmission Control Protocol)

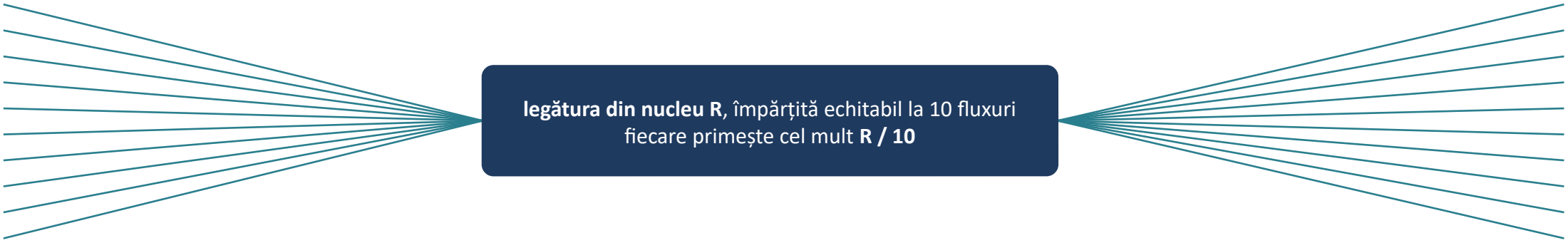
7.1 · Debitul capăt-la-capăt: decide legătura îngustă

debit = min(R_1, R_2, …, R_n)



O legătură partajată se împarte între fluxuri

1 Adesea limitează **legăturile de acces**; o poate face și un nucleu congestionat.



Ideea-cheie Debitul = cea mai mică rată pe care o poate obține acest flux pe oricare legătură a traseului, inclusiv pe cele partajate.

7.2 · Produsul lățime de bandă–întârziere: biți în zbor

fereastra ≥ BDP: conducta este plină — date înainte (sus), ACK-uri (acknowledgements, confirmări) înapoi (jos)



1 Gbit/s × 30 ms (București–Frankfurt și înapoi) = 30 Mbit ≈ **3,75 MB** în zbor

fereastra = 64 KB (limita inițială a TCP): aproape goală



64 KB la fiecare 30 ms ≈ **17,5 Mbit/s** pe un traseu de 1 Gbit/s

BDP = R × RTT — ca să țină traseul ocupat, un expeditor trebuie să aibă voie la un BDP de date neconfirmate

Ideea-cheie Un traseu plin cere cel puțin un BDP de date în zbor — revenim la el la ferestrele TCP.

TCP (Transmission Control Protocol) · RTT (round-trip time, timp dus-întors) · BDP (bandwidth–delay product, produsul lățime de bandă–întârziere) · ACK (acknowledgement, confirmare)

7 · Răspunsul la întrebarea de la început

1 MB: câțiva timpi dus-întors pentru stabilire și accelerare

voi



de ex. 3 RTT × 10 ms = 30 ms

vecinul



3 RTT × 300 ms = 900 ms

voi terminați primii

10 GB: decide rata legăturii înguste

voi



10 GB la 100 Mbit/s = 800 s

vecinul



10 GB la 1 Gbit/s ≈ 80 s

câștigă vecinul, de 10 ori

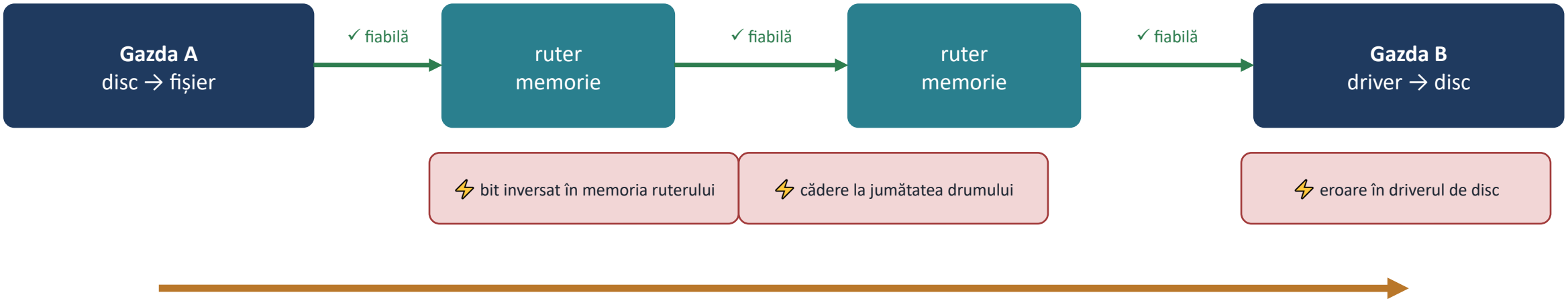
1 Cele 80 s ale vecinului cer o fereastră care acoperă BDP = 1 Gbit/s × 300 ms = **37,5 MB**.

Ideea-cheie Care legătură e „mai rapidă” depinde de mărimea a ceea ce trimiteți: întârzierea domină transferurile mici, debitul pe cele mari.

RTT (round-trip time, timp dus-întors) · BDP (bandwidth–delay product, produsul lățime de bandă–întârziere)

8 · Argumentul capăt-la-capăt: transferul atent de fișiere

O funcție poate fi implementată complet și corect doar cu cunoștințele și ajutorul aplicației din punctele terminale. În interiorul rețelei nu este niciodată **suficientă** — este doar o **îmbunătățire a performanței**.



1. Odată ce există verificarea capăt-la-capăt, fiabilitatea perfectă salt cu salt **nu mai este necesară pentru corectitudine**.

Ideea-cheie Doar capetele pot verifica ce le interesează pe capete: rețeaua poate ajuta, dar nu te poți baza pe ea.

8 · Când rețeaua ar trebui totuși să ajute: Wi-Fi

(a) recuperare lăsată doar pe seama TCP — pierderea este observată și reparată pe tot traseul



costă cel puțin un timp dus-întors capăt-la-capăt — zeci până la sute de ms

(b) Wi-Fi retransmite local, apoi TCP verifică în continuare capăt-la-capăt

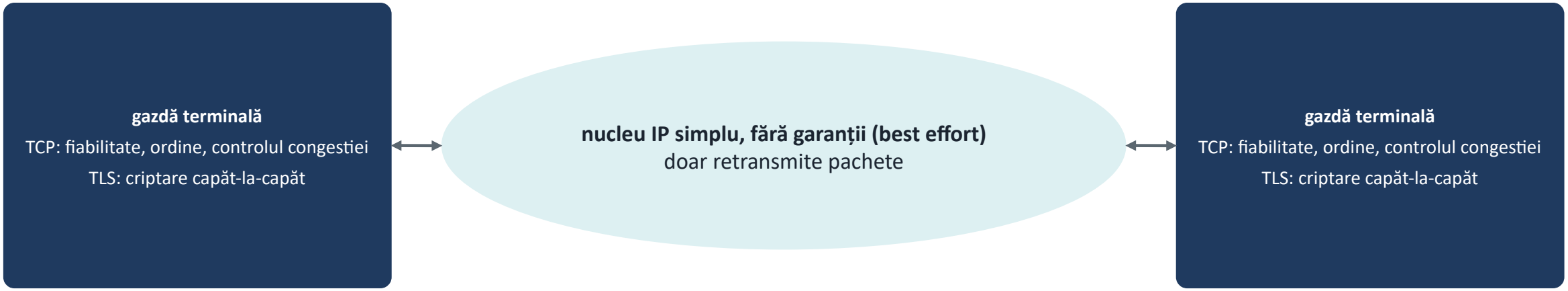


reparat în microsecunde, înainte ca TCP să observe măcar

Ideea-cheie Recuperarea salt cu salt este o îmbunătățire a performanței; corectitudinea vine tot din verificarea capăt-la-capăt.

TCP (Transmission Control Protocol)

8 · Consecințe în proiectarea Internetului



Echipamentele intermediare încalcă principiul — inspectează sau schimbă ce trece:

dispozitiv NAT | firewall | proxy transparent → protocol de transport nou, necunoscut → **blocat** ✗

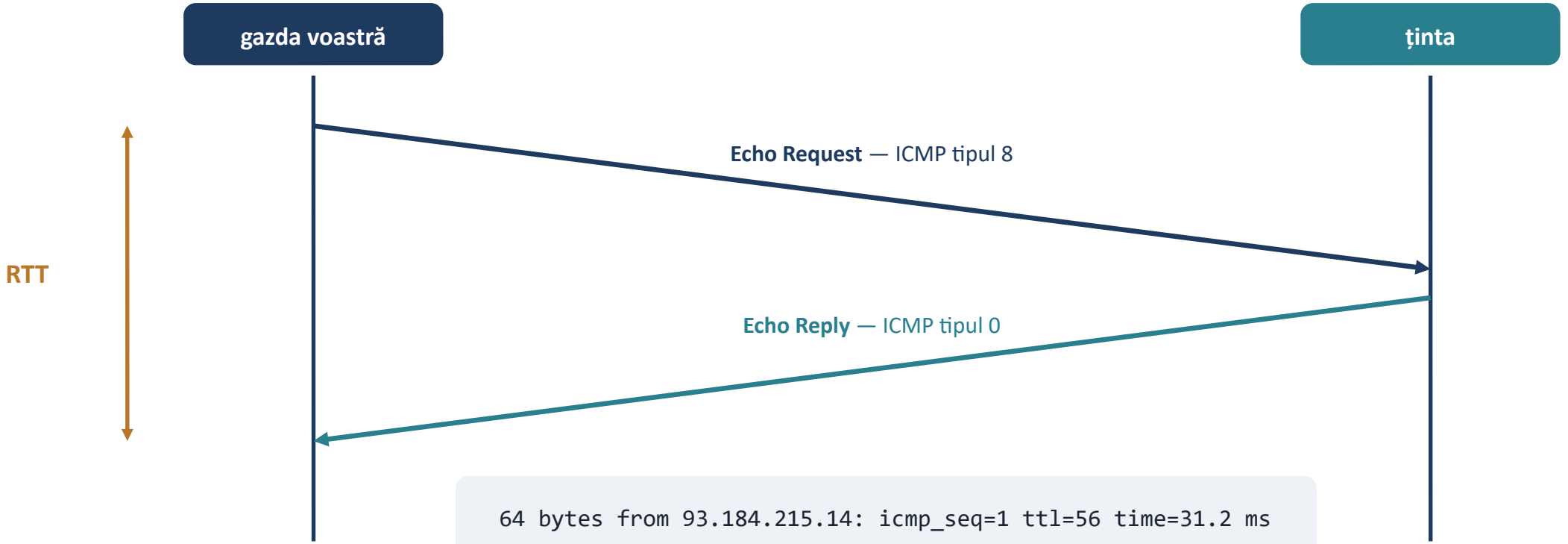
NAT = network address translation, translatarea adreselor de rețea

QUIC: arată ca UDP, antete criptate → **trece** ✓

Ideea-cheie Inteligența la margini, simplitatea în nucleu — echipamentele intermediare care încalcă asta fac Internetul greu de schimbat.

QUIC (protocol de transport peste UDP, User Datagram Protocol; numele inițial: Quick UDP Internet Connections) · TCP (Transmission Control Protocol) · UDP (User Datagram Protocol) · TLS (Transport Layer Security) · IP (Internet Protocol)

9 · Cum măsoară ping un timp dus-întors

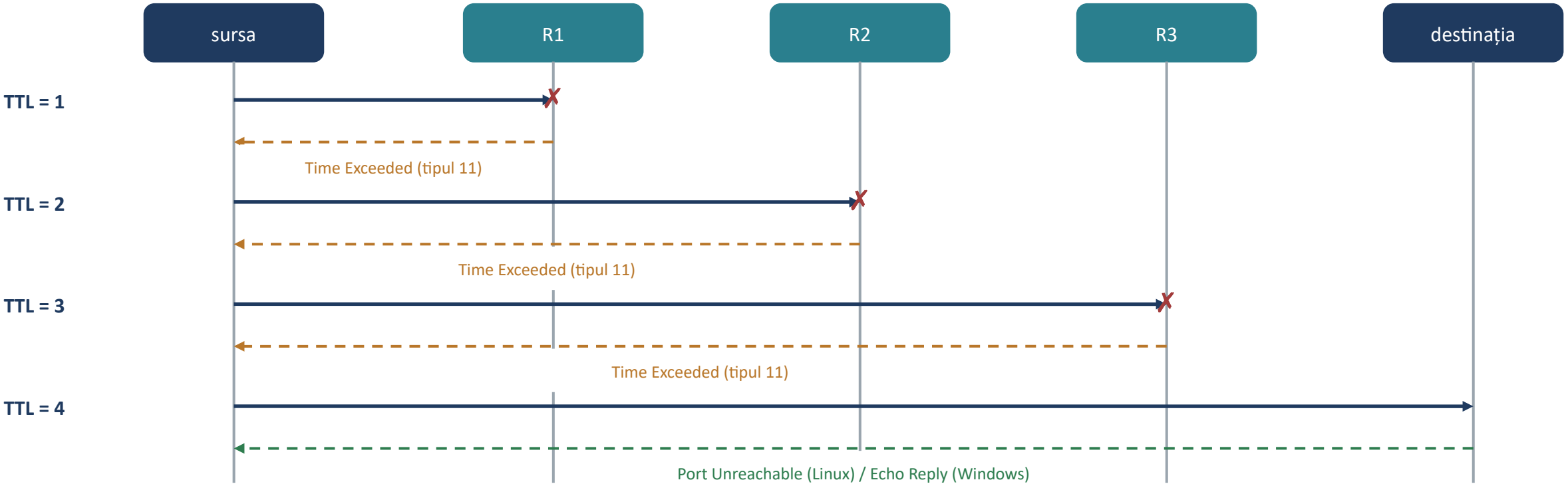


1 Include **transmisia, propagarea, așteptarea în coadă și procesarea — în ambele sensuri**. ICMP = Internet Control Message Protocol.

Ideea-cheie Un ping = un Echo Request + un Echo Reply = un timp dus-întors, cu toate cele patru întârzieri incluse.

RTT (round-trip time, timp dus-întors)

9 · traceroute: un instrument construit din câmpul TTL



traceroute pe Linux: sonde UDP spre porturi improbabile, de la 33434 în sus
destinația răspunde Port Unreachable

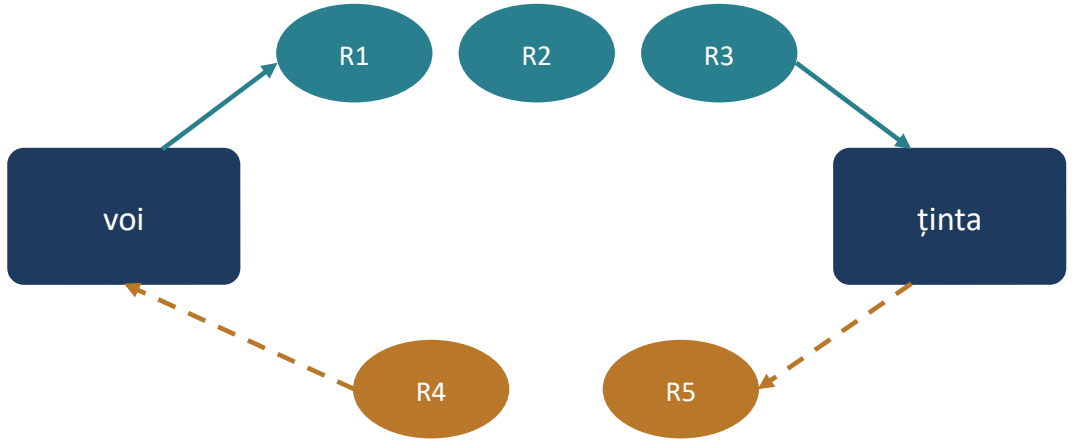
tracert pe Windows: sonde ICMP Echo Request
destinația răspunde Echo Reply

Ideea-cheie Van Jacobson, 1987: un instrument întreg construit dintr-un efect secundar al unui câmp de antet gândit pentru altceva.

ICMP (Internet Control Message Protocol) · UDP (User Datagram Protocol) · TTL (time to live, timp de viață)

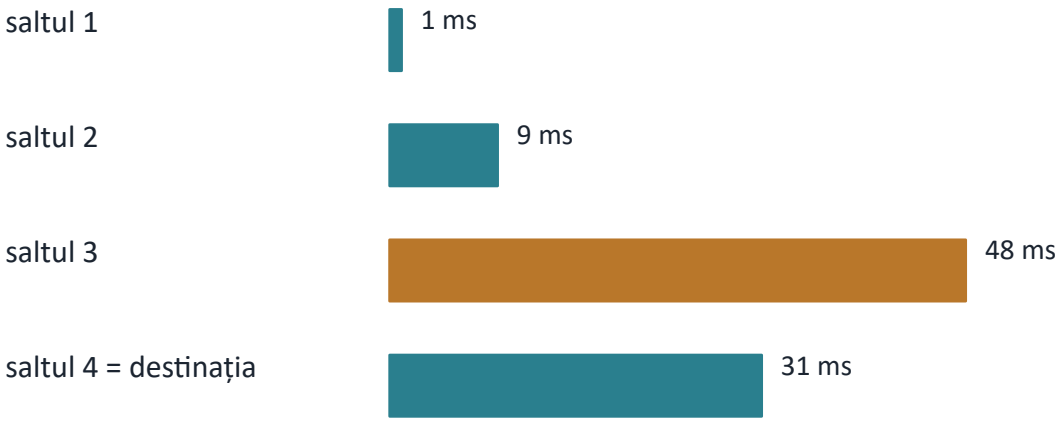
9 · Două precauții la citirea cifrelor

1 · Traseele pot fi asimetrice



Răspunsurile se pot întoarce pe alt drum: un RTT nu poate spune în ce parte se află întârzierea.

2 · Ruterele răspund ultimele



Ilustrație: saltul 3 își generează încet răspunsurile ICMP, așa că pare mai lent decât destinația din spatele lui.

Ideea-cheie Citiți traceroute cu atenție: un salt intermediar lent poate fi doar un ruter ocupat care răspunde ultimul la ICMP.

ICMP (Internet Control Message Protocol) · RTT (round-trip time, timp dus-întors)

Rezumatul Cursului 2

Cum este împărțită munca

Niveluri
serviciu · interfață · protocol · entități pereche

→

Încapsulare
antete la coborâre, demultiplexare la urcare: EtherType, Protocol, port

→

Modele
OSI 7 · TCP/IP 4 (5) · IP este talia îngustă

→

Standarde
IETF scrie RFC-uri · IANA înregistrează · RIPE NCC pentru România

Cât de bine funcționează

Cozi
ρ = La/R; peste 1, nelimitată; întârzierea explodează lângă 1; bufferbloat

→

Debit
legătura îngustă, cu tot cu partajare · BDP = R × RTT

→

Capăt-la-capăt
funcțiile care au nevoie de capete țin de capete

Ideea-cheie Structură și performanță: nivelurile fac rețeaua construibilă; cozile, legăturile înguste și capetele decid cât de bine funcționează.

RIPE NCC (Réseaux IP Européens Network Coordination Centre) · IETF (Internet Engineering Task Force) · IANA (Internet Assigned Numbers Authority) · TCP (Transmission Control Protocol) · RTT (round-trip time, timp dus-întors) · OSI (Open Systems Interconnection) · BDP (bandwidth–delay product, produsul lățime de bandă–întârziere) · RFC (Request for Comments) · IP (Internet Protocol)

Întrebări de recapitulare: unde este desenat fiecare răspuns

1 un serviciu, o interfață și un protocol la alt nivel

→ 2 · Serviciu, interfață, protocol

2 la ce protocoale participă ruterul?

→ 2.1 · Comunicarea între pereche

3 trei câmpuri care livrează un cadru unui navigator

→ 3 · Demultiplexarea

4 de ce se schimbă MAC-ul destinație și IP-ul nu

→ 3.1 · Spre ce indică MAC-ul

5 niveluri OSI fără corespondent TCP/IP

→ 4.2 · Trei modele

6 de ce talia îngustă ușurează aplicațiile, dar îngheață IP

→ 4.3 · Talia îngustă

7 „60 % folosit — 40 % rezervă, nu trebuie mărită”?

→ 6.2 · Curba M/M/1

8 de ce un buffer mai mare nu vindecă pierderile

→ 6.3 · Bufferbloat

9 argumentul capăt-la-capăt + un ajutor din rețea

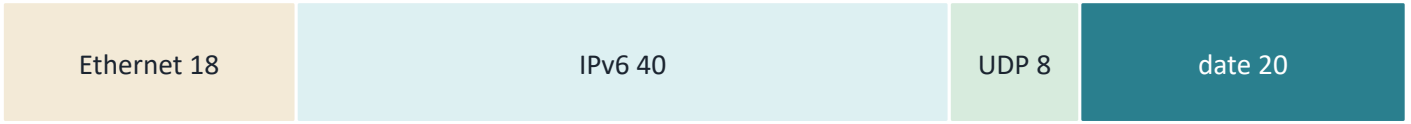
→ 8 · Capăt-la-capăt, Wi-Fi

Ideea-cheie Răspundeți la fiecare întrebare redesenând din memorie schema ei.

M/M/1 (sosiri markoviene / durate de servire markoviene / 1 server) · TCP (Transmission Control Protocol) · OSI (Open Systems Interconnection) · MAC (Media Access Control) · IP (Internet Protocol)

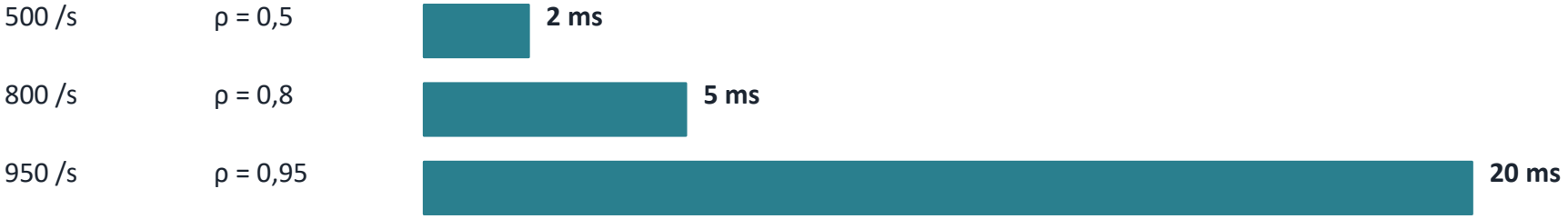
Exercițiile 1–3, rezolvate ca scheme

1 · Mesaj de 20 de octeți peste UDP și IPv6, pe Ethernet

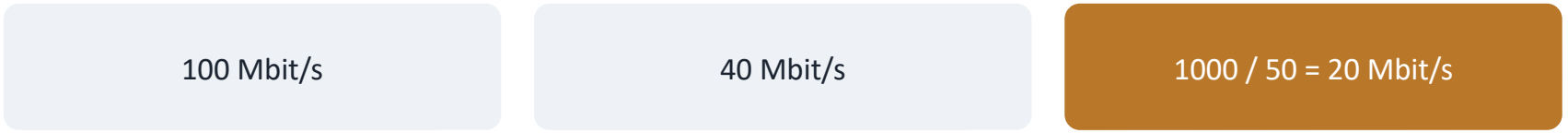


86 de octeți pe cadru → **20 / 86 ≈ 23 %** date

2 · Pachete de 1000 de octeți, legătură de 8 Mbit/s: L/R = 1 ms, ρ = 1 la 1000 de pachete/s



3 · legături de 100 Mbit/s, 40 Mbit/s și 1 Gbit/s, cea din urmă partajată de 50 de transferuri



→ min = **20 Mbit/s**

Ideea-cheie Desenați cadrul, curba, traseul — iar calculul vine de la sine.

IPv6 (Internet Protocol version 6) · UDP (User Datagram Protocol)

Exercițiile 4–5, rezolvate ca scheme

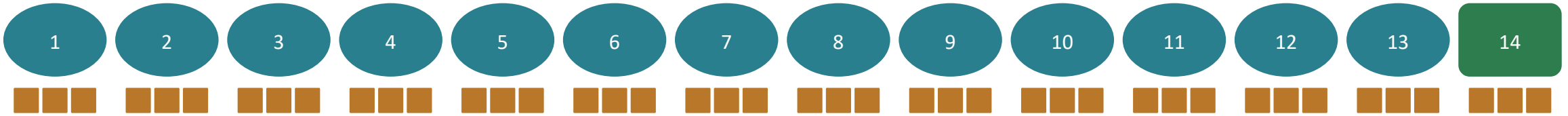
4 · Traseu transatlantic de 10 Gbit/s, RTT 80 ms

Europa

10^{10} bit/s × 0,08 s = 8×10^{8} biți = **100 MB** în zbor

America

5 · traceroute, 3 sonde pe salt, destinația la 14 salturi



14 × 3 = 42 de sonde
salturile 1–13 răspund **Time Exceeded**

saltul 14 (destinația) răspunde
Port Unreachable (Linux) sau Echo Reply (tracert pe Windows)

Ideea-cheie BDP = rată × timp dus-întors; un traceroute costă sonde × salturi și doar ultimul salt răspunde altfel.

RTT (round-trip time, timp dus-întors) · BDP (bandwidth–delay product, produsul lățime de bandă–întârziere)

Lecturi, laborator și cursul următor

Obligatoriu

Kurose și Ross, *Computer Networking: A Top-Down Approach*, ed. a 8-a, cap. 1: întârziere, pierderi, debit; niveluri de protocol

Recomandat

Saltzer, Reed, Clark, „End-to-End Arguments in System Design”, ACM (Association for Computing Machinery) *Transactions on Computer Systems*, 1984

Peterson și Davie, cap. 1 · RFC 1122, § 1.1 · Gettys și Nichols, „Bufferbloat”, 2012

Laboratorul 2

capturați propria cerere web, deschideți fiecare antet, verificați adresele MAC și IP, măsurați un RTT din stabilirea conexiunii TCP

Cursul următor

nivelul aplicație: arhitecturi, socket-uri, Web-ul și HTTP/1.1

Ideea-cheie Citiți capitolul 1, apoi revedeți în Wireshark fiecare schemă din acest curs.

HTTP (HyperText Transfer Protocol) · TCP (Transmission Control Protocol) · RTT (round-trip time, timp dus-întors) · MAC (Media Access Control) · RFC (Request for Comments) · IP (Internet Protocol)

Partea B · Seminarul 2: antete, cozi și debit

A · Cursul 2
Arhitectură și performanță

→

B · Seminarul 2
Antete, cozi, debit

→

C · Laboratorul 2 · Linux
Wireshark și tcpdump

→

D · Laboratorul 2 · Windows
Wireshark și pktmon

În această parte

A · Problema 1: ce nivel?

B · Problema 2: decodificarea manuală a unui cadru

C · Problema 3: cât din legătură este util?

D · Problemele 4–5: intensitatea traficului, legea lui Little

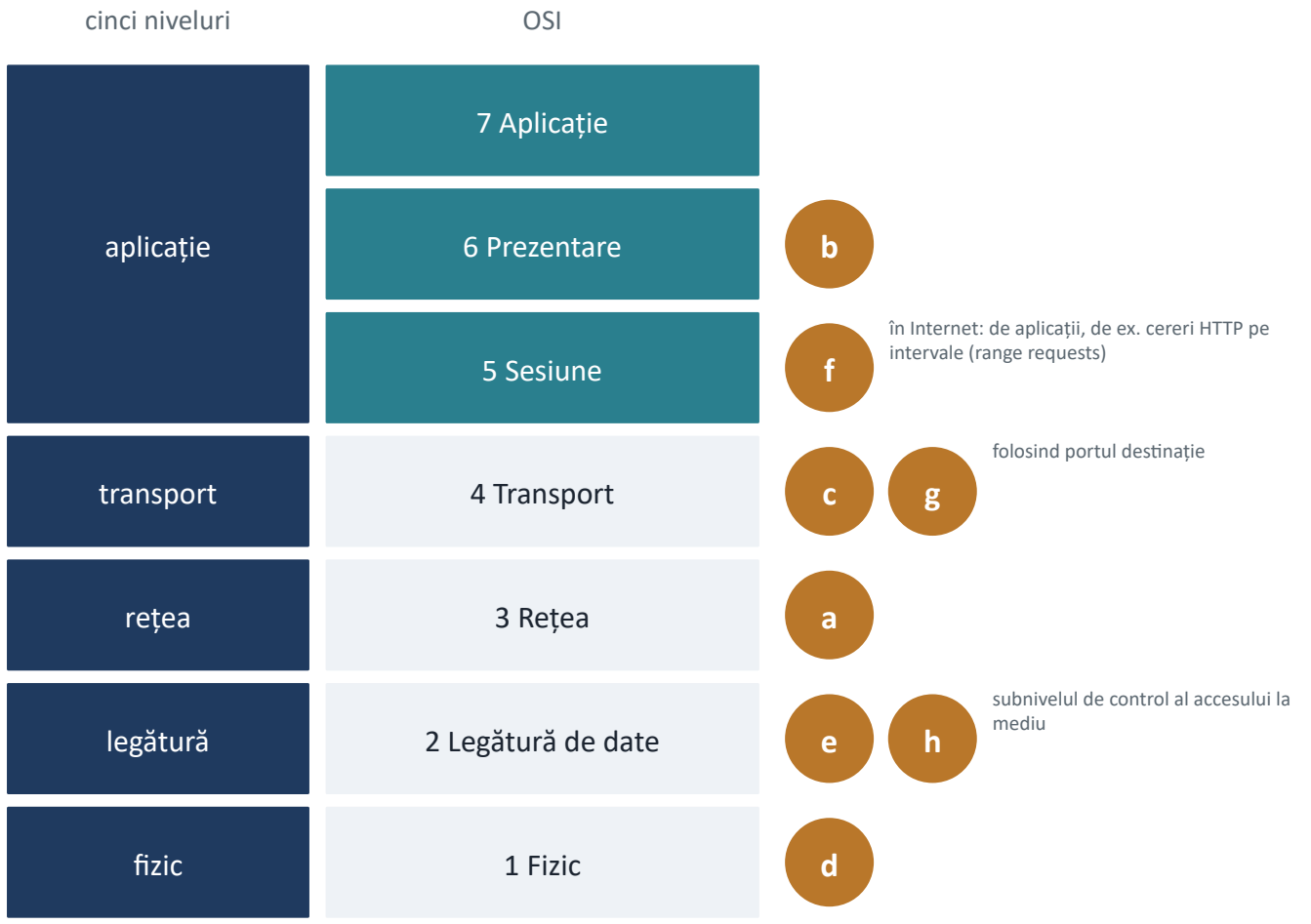
E · Problemele 6–7: legături înguste, produsul lățime de bandă–întârziere

F · Problema 8: argumentul capăt-la-capăt

Tema

Problema 1 · Ce nivel? Modelul cu cinci niveluri și OSI

- **a** alegerea legăturii de ieșire pentru un pachet spre altă țară
- **b** conversia unui document între codificări de caractere
- **c** retransmiterea unui segment al cărui ACK nu a sosit
- **d** reprezentarea unui 1 ca tranziție de tensiune
- **e** detectarea unui cadru corupt pe o legătură Wi-Fi
- **f** reluarea unei descărcări de unde s-a oprit
- **g** livrarea unui segment procesului potrivit
- **h** decizia care gazdă de pe un cablu partajat poate transmite



Ideea-cheie Munca nivelurilor prezentare și sesiune există și în Internet — doar că se face în interiorul aplicațiilor.

HTTP (HyperText Transfer Protocol) · OSI (Open Systems Interconnection) · ACK (acknowledgement, confirmare)

Problema 2 · Un cadru în hexazecimal, colorat pe antete

0000	52	54	00	12	34	01	52	54	00	3a	7f	10	08	00	45	00
0010	00	3d	1c	46	40	00	40	11	06	d5	0a	14	03	39	0a	14
0020	00	35	c8	22	00	35	00	29	96	d2	3b	7a	01	00	00	01
0030	00	00	00	00	00	00	03	77	77	77	07	65	78	61	6d	70
0040	6c	65	03	6f	72	67	00	00	01	00	01					

Ethernet
octeții 0x00–0x0d · 14 B

IPv4
0x0e–0x21 · 20 B

UDP
0x22–0x29 · 8 B

antet DNS
0x2a–0x35 · 12 B

întrebarea DNS
0x36–0x4a · 21 B

1 Întrebări: adresele MAC și tipul sarcinii utile · versiunea IP, lungimi · TTL, protocol, adrese · porturi și aplicație · numele căutat · al cui MAC?

Ideea-cheie Înainte de decodificare, tăiați octeții în antete: 14 + 20 + 8, iar tot ce urmează sunt date de aplicație.

IPv4 (Internet Protocol version 4) · UDP (User Datagram Protocol) · DNS (Domain Name System) · MAC (Media Access Control) · TTL (time to live, timp de viață) · IP (Internet Protocol)

Problema 2 · Harta câmpurilor pentru decodificare

Antetul Ethernet (14 octeți)

MAC destinație 6 B	MAC sursă 6 B	EtherType 2 B

Antetul UDP (8 octeți)

port sursă	port destinație
lungime	sumă de control

primul octet IPv4 = două jumătăți
cei 4 biți superiori = versiunea · cei 4 biți inferiori = lungimea antetului în cuvinte de 32 de biți

Antetul IPv4 (20 de octeți, câte 4 pe rând)

vers. | lung. antet	tip serviciu	lungime totală	
identificare		indicatori + deplasament fragment	
TTL	protocol	suma de control a antetului	
adresa sursă			
adresa destinație			
octetul 0	octetul 1	octetul 2	octetul 3

Ideea-cheie Fiecare câmp are o dimensiune și o poziție fixe — de aceea antetele pot fi decodificate de hardware și de oameni.

IPv4 (Internet Protocol version 4) · UDP (User Datagram Protocol) · MAC (Media Access Control) · TTL (time to live, timp de viață)

Problema 2 · Răspunsurile 1–3: Ethernet și IPv4 decodificate

MAC dest. 52:54:00:12:34:01	**MAC sursă** 52:54:00:3a:7f:10	**08 00** → IPv4

45 v4 · 5 × 4 = 20 B	**00** serviciu	**00 3d** = 61 de octeți
1c 46 identificare		**40 00** DF (Don’t Fragment, nu fragmenta), deplasament 0
40 TTL = 64	**11** 17 = UDP	**06 d5** sumă de control
0a 14 03 39 → sursa **10.20.3.57**		
0a 14 00 35 → destinația **10.20.0.53**		

1. EtherType 0x0800: sarcina utilă este o datagramă **IPv4**.
2. Lungimea totală 61; cadrul afișat are 75 = **14 + 61** ✓ coerent.
3. TTL 64: o valoare inițială uzuală — datagrama **nu a trecut încă printr-un ruter**.

Ideea-cheie 0x45, 0x0800, 0x11: trei octeți care spun IPv4 cu antet de 20 de octeți, transportat de Ethernet, care transportă UDP.

IPv4 (Internet Protocol version 4) · UDP (User Datagram Protocol) · MAC (Media Access Control) · TTL (time to live, timp de viață)

Problema 2 · Răspunsurile 4–5: antetul UDP și numele DNS

c8 22 port sursă 51234 (dinamic → client)	**00 35** port dest. 53 (server DNS)	**00 29** lungime 41 = 8 + 33	**96 d2** sumă de control

După 12 octeți de antet DNS, întrebarea:

03	77	77	77	07	65	78	61	6d	70	6c	65	03	6f	72	67	00	00 01	00 01
3	w	w	w	7	e	x	a	m	p	l	e	3	o	r	g	sfârșit	tip A	clasa IN

→ **www.example.org**

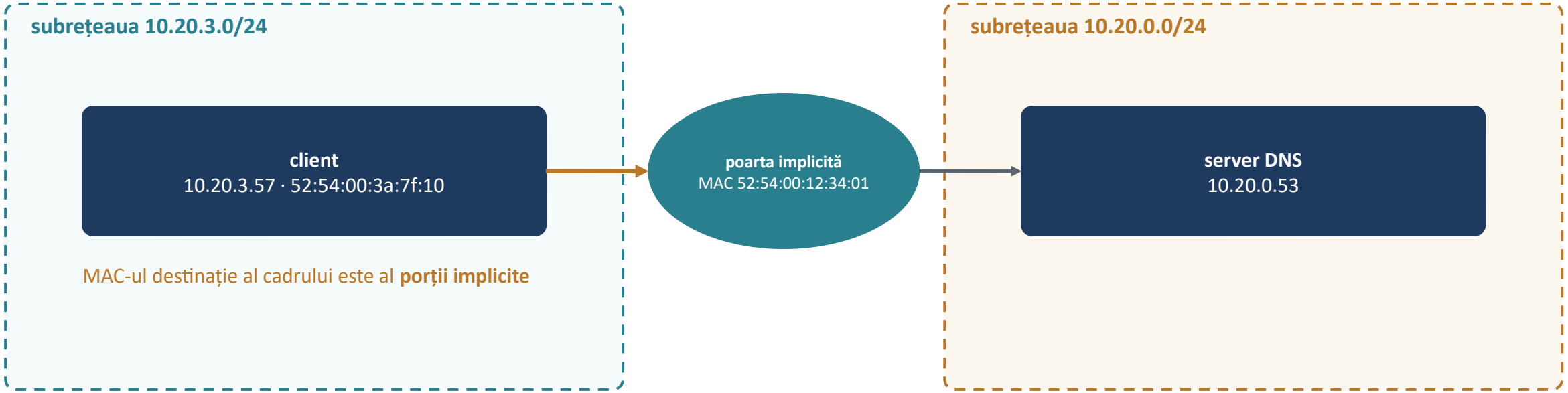
adresă IPv4, clasa Internet

1 Octeții chihlimbarii sunt **contoare de lungime** (3, 7, 3), apoi 0 încheie numele. 0x77 este codul ASCII (American Standard Code for Information Interchange) al lui „w”.

Ideea-cheie Lungimea UDP 41 = antet de 8 octeți + 33 de octeți DNS: 12 de antet, 17 de nume, 4 de tip și clasă.

IPv4 (Internet Protocol version 4) · UDP (User Datagram Protocol) · DNS (Domain Name System) · IN (Internet class, clasa Internet)

Problema 2 · Răspunsul 6: este MAC-ul destinație al serverului?



10.20.3.57 față de **10.20.0**.53
primii 24 de biți diferă → nu e pe legătura mea → trimit la poarta implicită (default gateway)

verificați
`ip neigh` (Linux) · `arp -a` (Windows)
— exact ce faceți în Laboratorul 2

Ideea-cheie Un cadru pentru altă subrețea poartă întotdeauna MAC-ul porții implicite — antetul IP numește în continuare destinația reală.

DNS (Domain Name System) · MAC (Media Access Control) · IP (Internet Protocol)

Problema 3 · Un cadru complet ocupă pe fir 1538 de timpi-octet



8 + 14 + 1500 + 4 + 12 = 1538 de timpi-octet pentru un cadru de dimensiune maximă

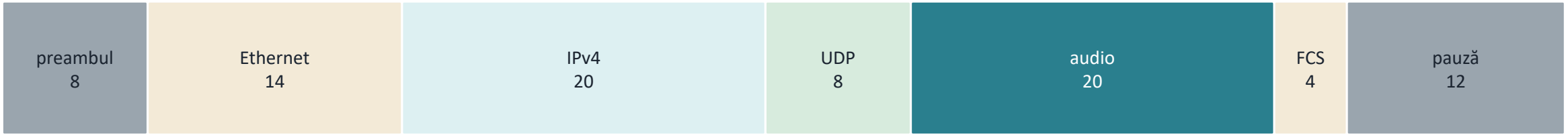
peste IPv4	IPv4 20	TCP 20	date **1460**	1460 / 1538 × 100 ≈ **94,9 Mbit/s**
peste IPv6	IPv6 40	TCP 20	date **1440**	1440 / 1538 × 100 ≈ **93,6 Mbit/s**

1 Rata maximă de date de aplicație pe un Ethernet de 100 Mbit/s — de aceea testele de viteză pe legături „de 100 Mbit/s” nu arată niciodată 100.

Ideea-cheie Încadrarea, antetele și pauzele iau aproximativ 5 % din legătură, chiar și cu cele mai mari cadre.

IPv4 (Internet Protocol version 4) · IPv6 (Internet Protocol version 6) · TCP (Transmission Control Protocol) · FCS (frame check sequence, secvența de verificare a cadrului)

Problema 3 · Q4: eșantioanele vocale mici risipesc legătura



86 de timpi-octet pe fir; audio înseamnă 20 dintre ei → **20 / 86 ≈ 23 %**

Regula umpluturii (padding)

Ethernet completează până la 46 de octeți sarcinile utile mai scurte de 46.

Aici sarcina utilă are 48 de octeți: nu e nevoie de umplutură.

Ce fac codecurile reale

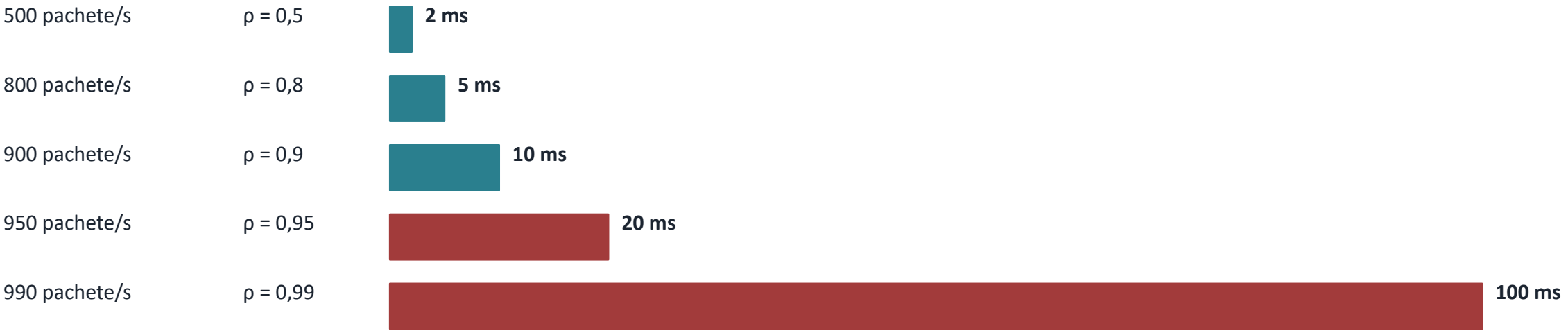
trimit eșantioane mai mari, mai rar:

cost suplimentar mai mic, cu prețul unei mici întârzieri în plus până se umple un eșantion

Ideea-cheie Mesajele mici plătesc fiecare antet: trei sferturi din timpul acestei legături nu transportă audio.

IPv4 (Internet Protocol version 4) · UDP (User Datagram Protocol) · FCS (frame check sequence, secvența de verificare a cadrului)

Problema 4 · Intensitatea traficului pe o legătură de 10 Mbit/s

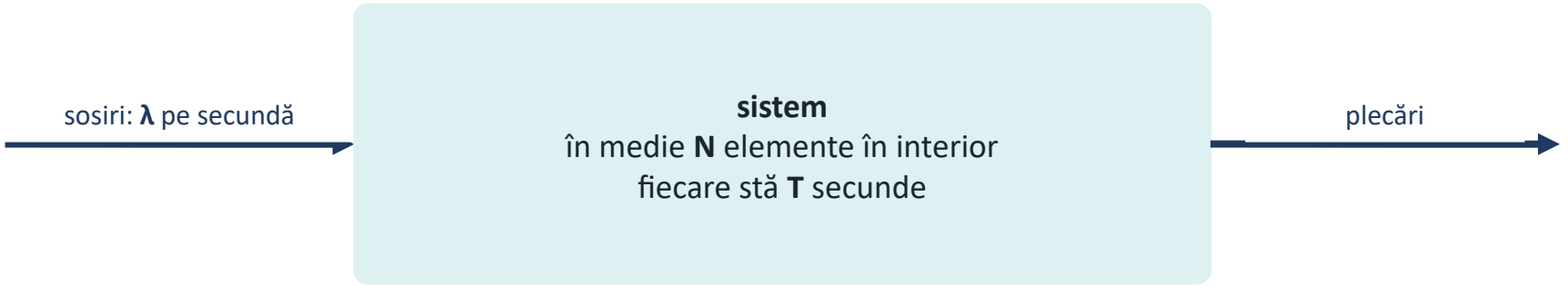


Q3 · traficul crește cu 5 % pe lună; azi legătura este la 80 %



Ideea-cheie Operatorul care așteaptă „plinul” așteaptă prea mult: aici dublarea întârzierii vine în mai puțin de trei luni.

Problema 5 · Legea lui Little: N = λ × T



1 **memoria tampon a unui ruter**

N = 40 de pachete, λ = 2000 /s

T = N / λ = **20 ms**

2 **server web**

λ = 300 de cereri/s, T = 50 ms

N = 300 × 0,05 = **15 cereri**

3 **conexiune TCP**

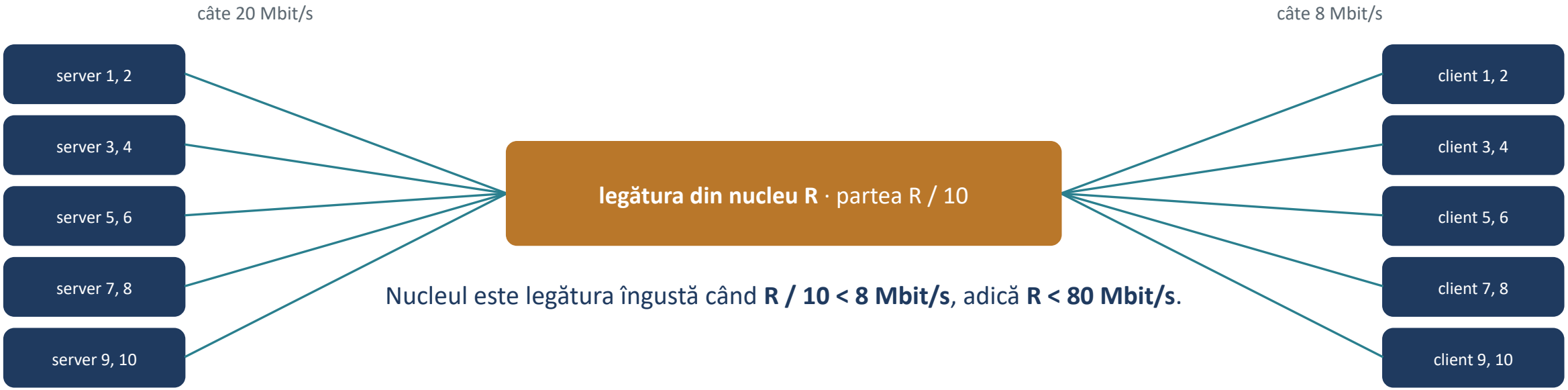
N = 64 KB în zbor, T = RTT = 20 ms

rata = N / T ≈ 3,3 MB/s ≈ **26 Mbit/s**

Ideea-cheie Legea lui Little leagă cele trei mărimi: același raționament (date în zbor = rată × timp) dă produsul lățime de bandă–întârziere.

TCP (Transmission Control Protocol) · RTT (round-trip time, timp dus-întors)

Problema 6 · Zece transferuri pe o legătură partajată

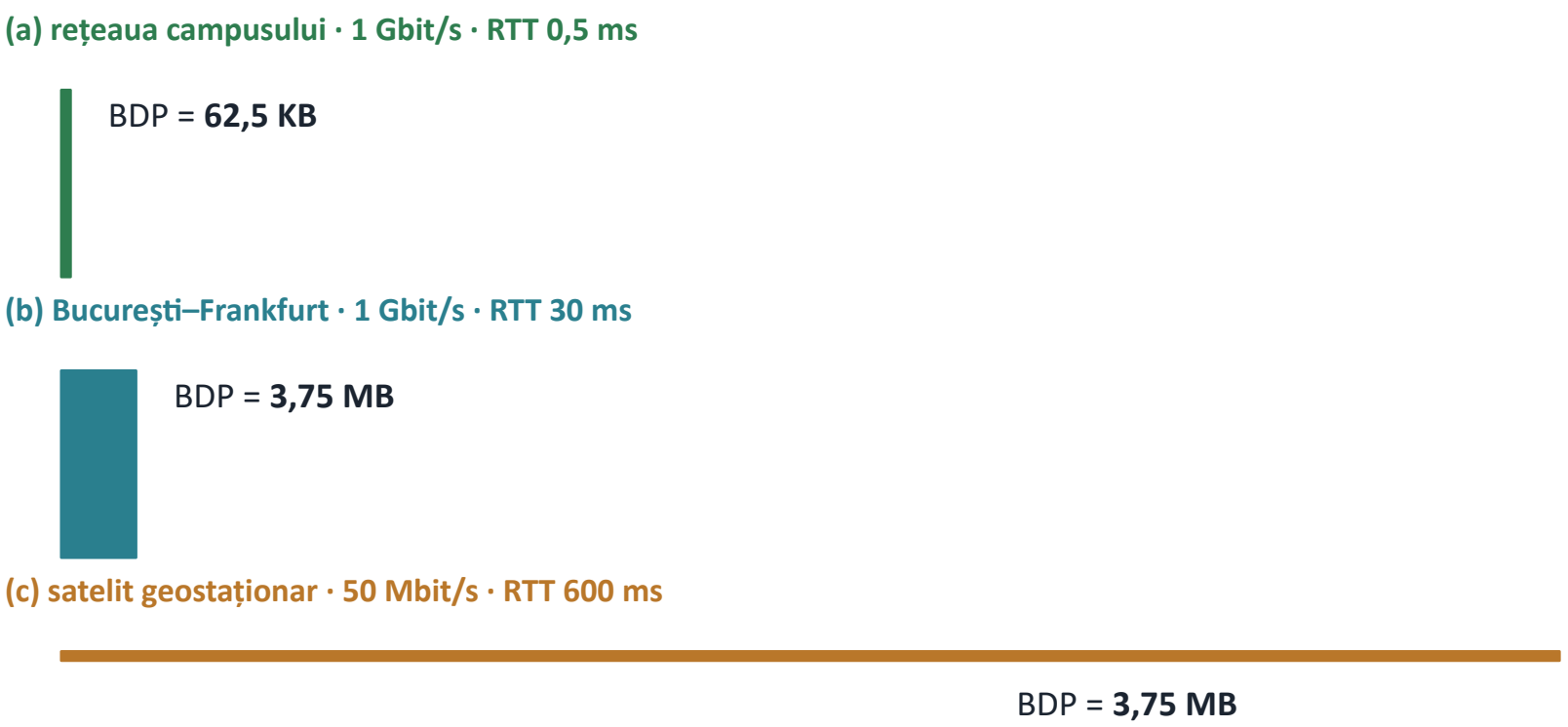


Nucleul este legătura îngustă când **R / 10 < 8 Mbit/s**, adică **R < 80 Mbit/s**.

R	min(20, 8, R/10)	cine limitează
1 Gbit/s	min(20, 8, 100) = **8 Mbit/s**	legătura de acces a clientului
50 Mbit/s	min(20, 8, 5) = **5 Mbit/s**	legătura partajată din nucleu

Ideea-cheie Pentru fiecare flux, calculați ce îi oferă fiecare legătură — câștigă cea mai mică, iar ea se poate muta când R se schimbă.

Problema 7 · Trei trasee și produsele lor lățime de bandă–întârziere



(b) și (c): **aceeași arie**
o legătură lentă prin satelit ține în zbor cât una terestră rapidă

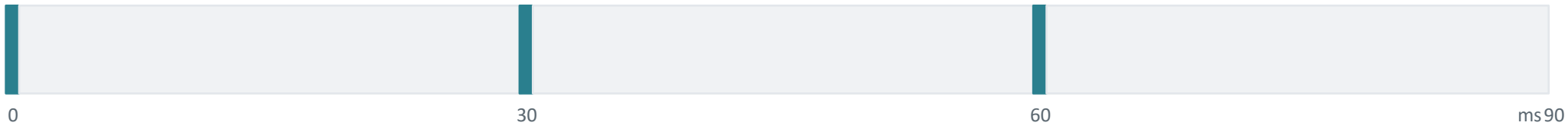
desenat la scară; (a) este o fâșie:
10^9 bit/s × 0,0005 s = 0,5 Mbit = 62,5 KB

Ideea-cheie BDP = rată × RTT: un traseu lung și unul rapid pot cere aceeași cantitate de date în zbor.

RTT (round-trip time, timp dus-întors) · BDP (bandwidth–delay product, produsul lățime de bandă–întârziere)

Problema 7 · Q2–Q3: o fereastră de 64 KB pe traseul (b)

trei timpi dus-întors, la scară (RTT 30 ms)



fâșia turcoaz = 64 KB trimiși (≈ 0,5 ms) · gri = legătură liberă, se așteaptă confirmarea

65 535 × 8 biți / 0,03 s ≈ **17,5 Mbit/s**
sub 2 % din legătura de 1 Gbit/s

ca să umpleți traseul: fereastra ≥ **un BDP = 3,75 MB**

1 Antetul TCP inițial putea exprima cel mult 64 KB; opțiunea de scalare a ferestrei (**window scale**, RFC 7323) a fost adăugată ca să elimine această limită.

Ideea-cheie O fereastră mai mică decât BDP lasă legătura liberă cea mai mare parte a timpului, oricât de rapidă ar fi.

TCP (Transmission Control Protocol) · RTT (round-trip time, timp dus-întors) · BDP (bandwidth–delay product, produsul lățime de bandă–întârziere) · RFC (Request for Comments)

Problema 8 · Patru propuneri judecate prin argumentul capăt-la-capăt

Ruterele verifică fiecare sumă de control TCP și aruncă segmentele corupte, deci gazdele nu mai trebuie să verifice.	✗	**incoerentă ca înlocuitor** corupția apare și în interiorul ruterelor și al gazdelor; doar o verificare capăt-la-capăt o prinde — iar în practică ruterele nu verifică sumele de control TCP, din cauza costului
Legătura radio a unui operator mobil retransmite local, de câteva ori, cadrele pierdute.	✓	**coerentă: o îmbunătățire a performanței** TCP verifică în continuare capăt-la-capăt; legătura doar recuperează mai repede pierderile radio frecvente
Proxy-ul unui ISP (Internet service provider, furnizor de servicii Internet) recomprimă toate imaginile descărcate de clienți.	!	**problematică** rețeaua modifică date pe care capetele nu au cerut să fie modificate; o imagine medicală nu tolerează asta — HTTPS anulează azi astfel de proxy-uri
Băncile criptează capăt-la-capăt cu TLS, deși rețeaua internă este criptată legătură cu legătură.	✓	**coerentă** doar capetele pot garanta confidențialitatea dintre ele; criptarea pe legătură lasă datele în clar în interiorul fiecărui dispozitiv

Ideea-cheie O poate face corect rețeaua singură? Dacă nu, o fac capetele — rețeaua doar o poate accelera.

HTTPS (HyperText Transfer Protocol Secure) · TCP (Transmission Control Protocol) · TLS (Transport Layer Security)

Tema

1 Exercițiile 1–5

de la sfârșitul Cursului 2 — rezolvați-le fără să vă uitați la răspunsuri

2 Decodificați propriul SYN

primii 54 de octeți ai unui segment TCP SYN (synchronize, sincronizare) capturat în Laboratorul 2, de mână, ca în Problema 2; verificați fiecare câmp în Wireshark

3 Citiți și reflectați

secțiunea 2 din Saltzer–Reed–Clark (transferul atent de fișiere); cinci rânduri despre dacă argumentul se aplică livrării e-mailului

Ideea-cheie Decodificarea unui cadru capturat de voi este cel mai bun test că harta câmpurilor v-a rămas în minte.

TCP (Transmission Control Protocol)

Partea C · Laboratorul 2 pe Linux

A · Cursul 2
Arhitectură și performanță

→

B · Seminarul 2
Antete, cozi, debit

→

C · Laboratorul 2 · Linux
Wireshark și tcpdump

→

D · Laboratorul 2 · Windows
Wireshark și pktmon

În această parte

Înainte de a începe · plan

1 · O primă captură: cele trei panouri

2 · Filtre de captură și filtre de afișare

3 · O cerere web, nivel cu nivel

4 · Întreaga conversație

5 · Măsurarea RTT din stabilirea conexiunii

6 · Traceroute la microscop

7 · Captură din linia de comandă: tcpdump, tshark

8 · HTTPS: ce rămâne vizibil

Dacă ceva nu funcționează

HTTPS (HyperText Transfer Protocol Secure) · RTT (round-trip time, timp dus-întors)

Înainte de a începe (Linux)

1 · instalare	2 · permiteți captura	3 · deconectare și reconectare	4 · verificare
`sudo apt install -y wireshark tshark tcpdump traceroute curl dnsutils`	răspundeți „Yes” pentru utilizatori fără drepturi de superutilizator · `sudo usermod -aG wireshark $USER`	noul grup se aplică la următoarea autentificare	`groups | grep wireshark`

Mediu
mașina virtuală a cursului (VM, virtual machine) sau Linux cu interfață grafică · în WSL 2 (Windows Subsystem for Linux): tcpdump acolo, fișierele deschise în Wireshark pe Windows

Numele interfeței
`ip -br link` → enp0s3, ens33, wlp2s0 … · mai jos, **eth0** ține locul interfeței voastre

Reguli de conduită
doar propria mașină, doar propriul trafic — capturarea traficului altora este neetică și adesea ilegală

De predat
răspunsurile la **Q1–Q16** + captura **web.pcapng** din Sarcina 3 · valorile afișate aici sunt ilustrații

Ideea-cheie Captura cere permisiune de două ori: de la sistem (grupul wireshark) și de la oamenii al căror trafic este.

VM (virtual machine, mașină virtuală)

Laboratorul 2 (Linux) · cele două ore

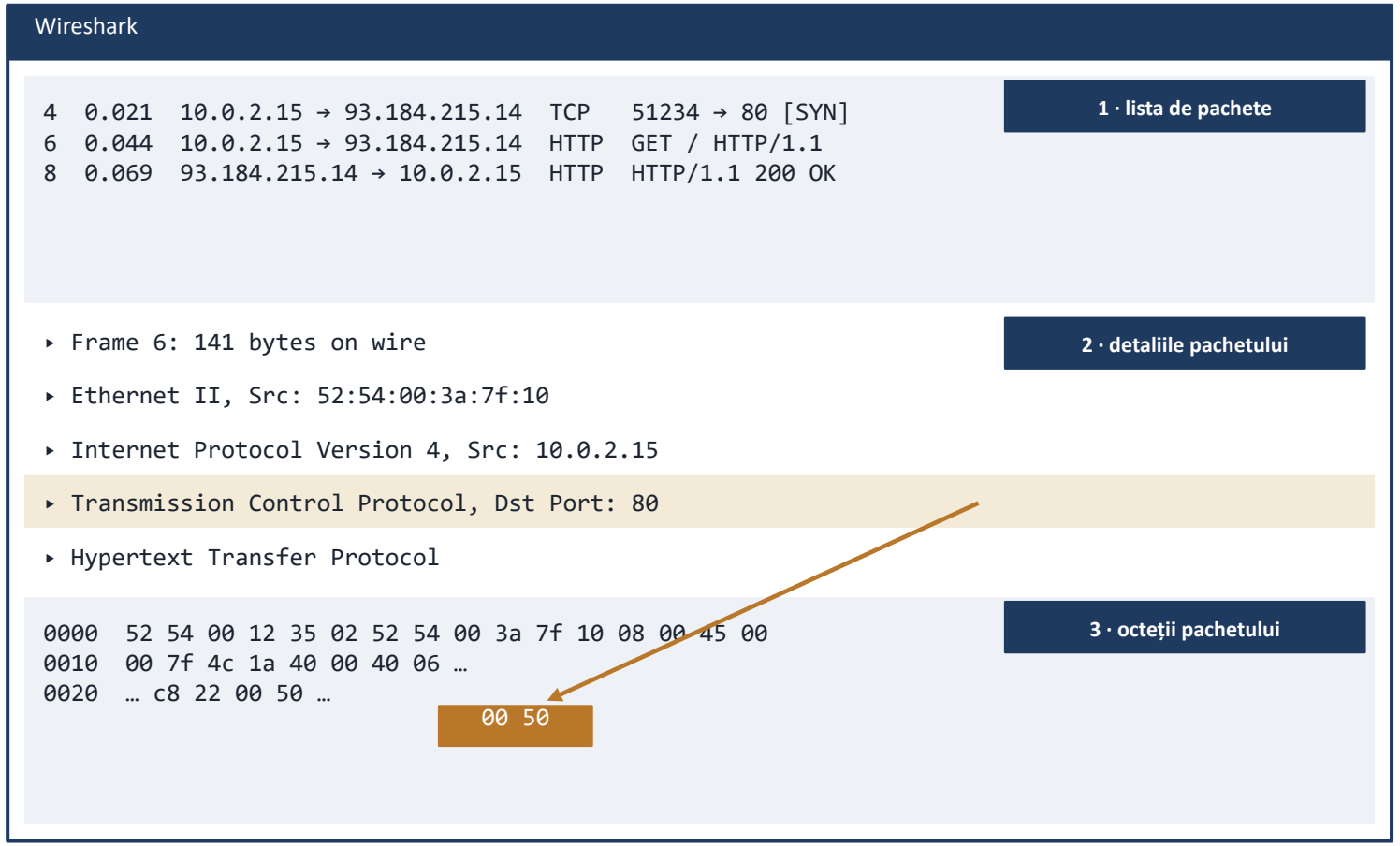


1. O primă captură și un tur al Wireshark
2. Filtre de captură și filtre de afișare
3. O cerere web, nivel cu nivel
4. Întreaga conversație: fluxuri, statistici, graful fluxului
5. Măsurarea RTT (round-trip time, timp dus-întors) din stabilirea conexiunii
6. Traceroute la microscop
7. Captură din linia de comandă: tcpdump și tshark
8. HTTPS: ce rămâne vizibil

Ideea-cheie Sarcina 3 este inima laboratorului: o cerere web deschisă la fiecare nivel.

HTTPS (HyperText Transfer Protocol Secure)

Sarcina 1 · Cele trei panouri ale Wireshark




```
curl -s http://example.org/ > /dev/null
```

← (informații de captură)

← legătură de date

← rețea

← transport

← aplicație

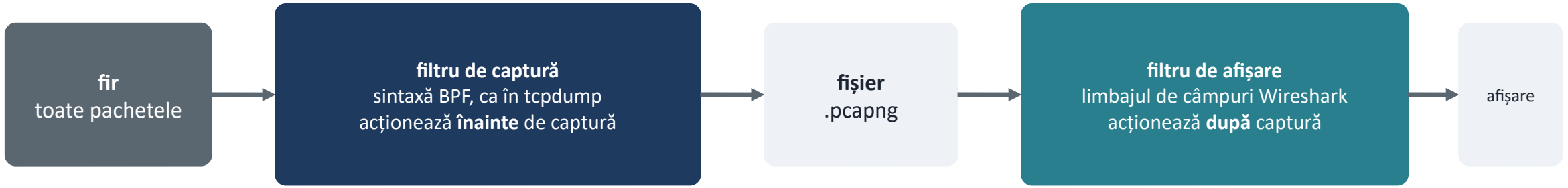
Q1 · câte pachete, câte de la curs? ce altceva mai trimite mașina voastră?

Q2 · asociați liniile de detaliu celor cinci niveluri

Ideea-cheie Lista = o linie pe pachet; detaliile = o linie pe nivel; octeții = cadrul brut. Clicul le leagă între ele.

SYN (synchronize, sincronizare)

Sarcina 2 · Filtre de captură și filtre de afișare




```
host example.org · port 53
tcp port 80 · icmp
port 53 or port 80 ← folosiți-l pe acesta
```

```
dns · http · tcp.port == 80 · icmp
ip.addr == 10.20.0.53
tcp.flags.syn == 1
```

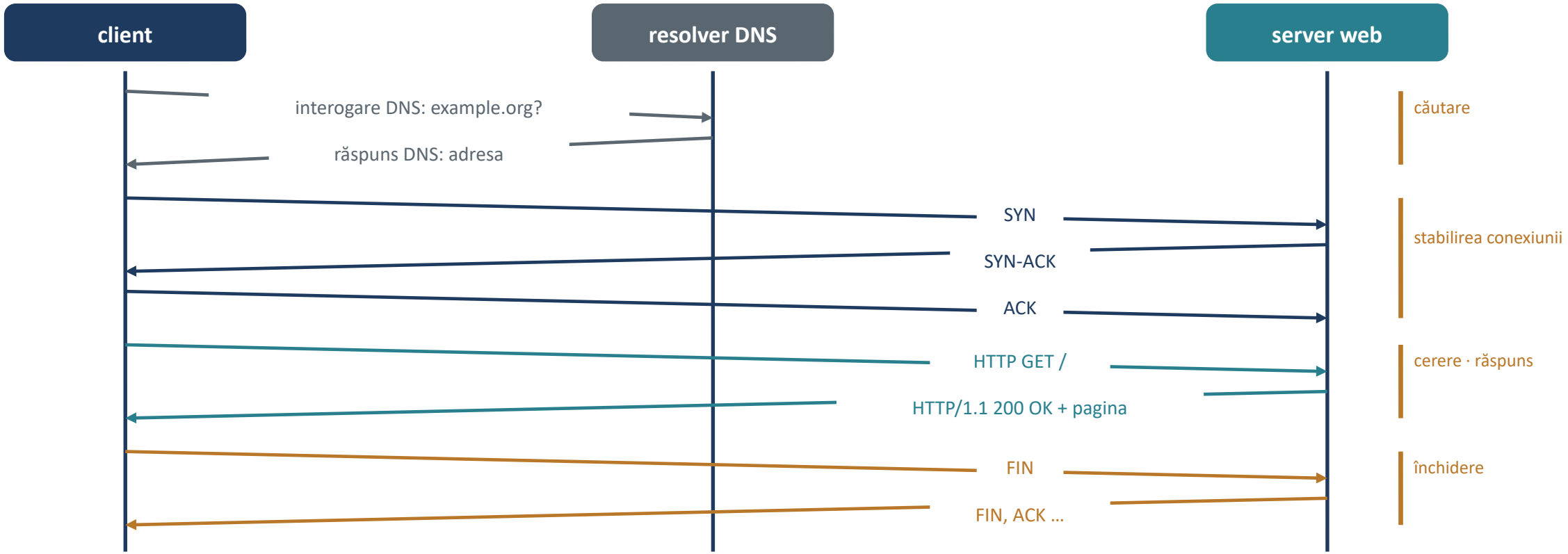
1 **Q3** · filtre de afișare pentru: doar răspunsurile DNS (indiciu: `dns.flags.response`) · doar pachetele trimise **de** mașina voastră.

Ideea-cheie Aceeași idee, două limbaje: BPF (Berkeley Packet Filter) înainte de înregistrare, câmpurile Wireshark după.

DNS (Domain Name System)

Sarcina 3 · O cerere web, pachet cu pachet

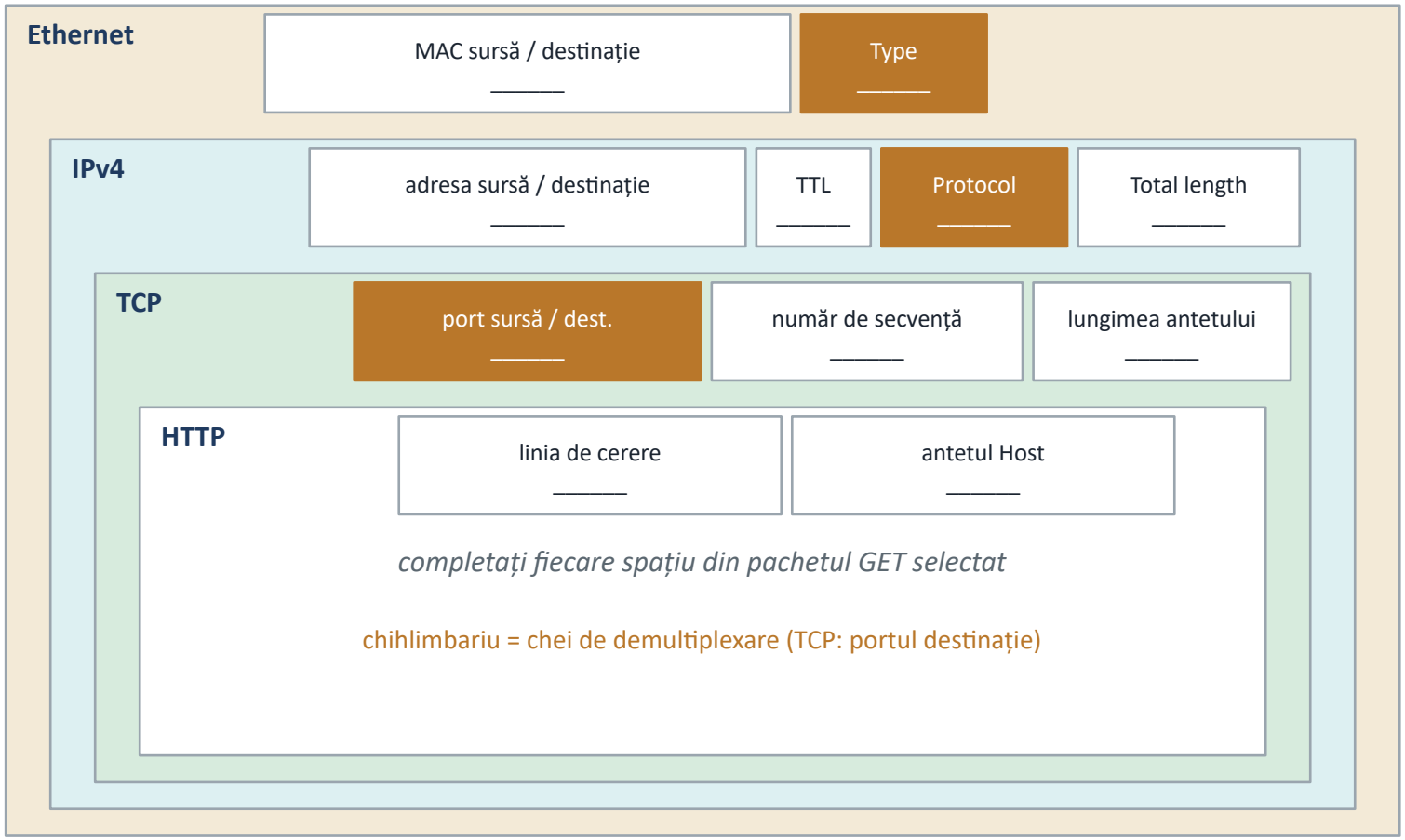
```
resolvectl flush-caches   →   captură   →   curl -s http://example.org/ > /dev/null   →   stop   →   File → Save As… web.pcapng
```



Ideea-cheie O pagină mică: căutare, stabilirea conexiunii, cerere, răspuns, închidere — găsiți-le în această ordine.

HTTP (HyperText Transfer Protocol) · DNS (Domain Name System) · SYN (synchronize, sincronizare) · ACK (acknowledgement, confirmare) · FIN (finish, încheiere)

Sarcina 3 · Pachetul GET, nivel în nivel



1 **Q4** · câmpurile chihlimbarii sunt cheile de demultiplexare. Cu EtherType 0x86DD, sarcina utilă ar merge la **IPv6**.

2 **Q7** · antete înainte de primul octet HTTP: 14 + 20 + 20 = **54** de octeți (Cursul 2, § 3.1). Adesea vedeți **66**: Linux adaugă 12 octeți de opțiuni TCP (marcaje de timp, timestamps).

Ideea-cheie Fiecare câmp desenat la curs se află în acest singur pachet — cele chihlimbarii arată cum a ajuns la curl.

HTTP (HyperText Transfer Protocol) · IPv4 (Internet Protocol version 4) · IPv6 (Internet Protocol version 6) · TCP (Transmission Control Protocol) · MAC (Media Access Control) · TTL (time to live, timp de viață)

Sarcina 3 · Q5–Q6: al cui MAC, câte rutere?




```
GET: dst MAC = 52:54:00:12:35:02
```

```
$ ip neigh → 10.0.2.2 lladdr 52:54:00:12:35:02
```

Q5 · aceeași adresă: cadrul merge doar la următorul salt.

Q6 · numărarea ruterelor din TTL-ul răspunsului

64	128	255
Linux, majoritatea Unix	Windows	multe rutere

TTL-ul răspunsului 52 → următoarea valoare inițială de deasupra este 64 → 64 − 52 = **12 rutere**

1 Drumul dus și cel întors pot avea lungimi diferite: dirijarea se alege separat în fiecare sens, de rețele diferite.

Ideea-cheie Adresa MAC pe care o vedeți este a porții implicite; TTL-ul primit este o valoare inițială minus ruterele de pe drumul de întoarcere.

MAC (Media Access Control) · TTL (time to live, timp de viață) · PC (personal computer, calculator personal)

Sarcina 4 · Patru perspective asupra aceleiași conversații

Follow → TCP Stream

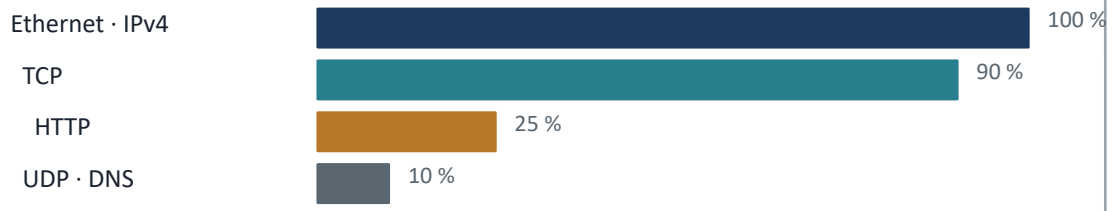
datele de aplicație din ambele sensuri, colorate

```
GET / HTTP/1.1  ·  Host: example.org
```

```
HTTP/1.1 200 OK  ·  Content-Type: text/html
```

Statistics → Protocol Hierarchy

cum se împart octeții între protocoale



Statistics → Conversations

fiecare pereche de capete, octeții schimbați

adresa A ↔ adresa B	pachete	octeți
10.0.2.15 ↔ resolver	2	≈ 160
10.0.2.15 ↔ server	≈ 10	≈ 2 300

Statistics → Flow Graph

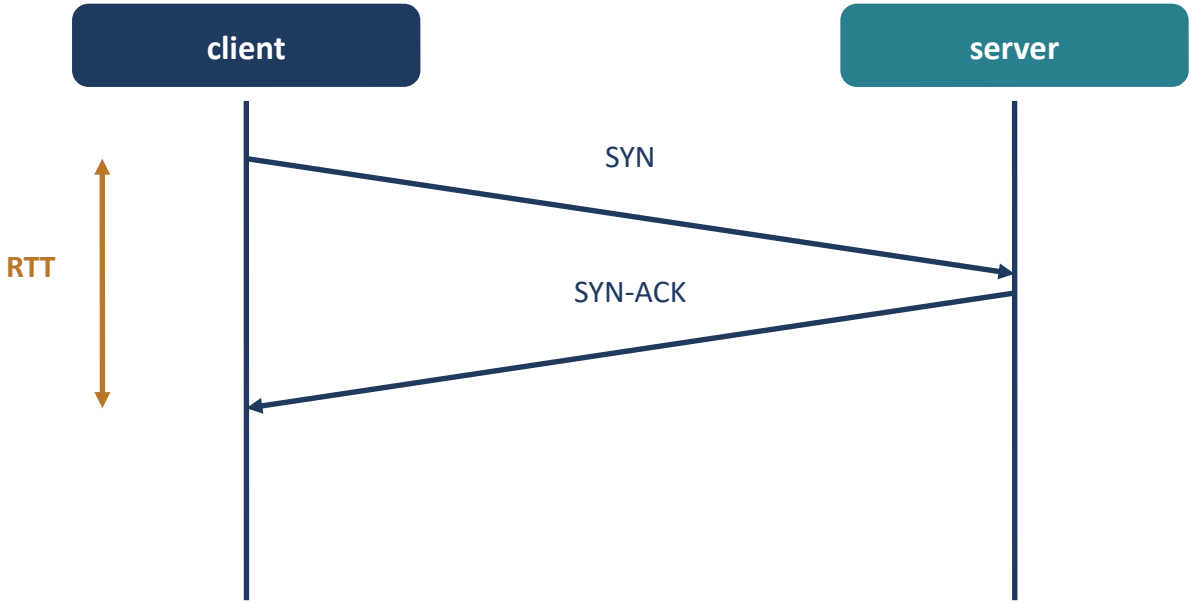
schimbul ca diagramă timp–secvență



Ideea-cheie Q8 · Lipiți graful fluxului și etichetați căutarea, stabilirea conexiunii, cererea, răspunsul, închiderea — apoi numărați pachetele.

HTTP (HyperText Transfer Protocol) · IPv4 (Internet Protocol version 4) · TCP (Transmission Control Protocol) · UDP (User Datagram Protocol) · DNS (Domain Name System) · SYN (synchronize, sincronizare) · ACK (acknowledgement, confirmare)

Sarcina 5 · Măsurarea RTT din stabilirea conexiunii



No.	Time	Info
4	0.000	51234 → 80 [SYN]
5	0.031	80 → 51234 [SYN, ACK]

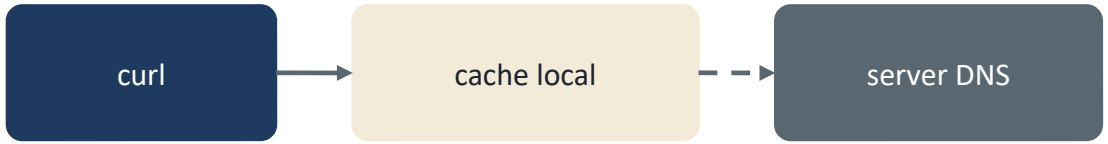
Q9 · comparați trei numere

timpul SYN → SYN-ACK · `ping -c 5 example.org`

câmpul `tcp.analysis.ack_rtt` (analiza SEQ/ACK)

diferențe: pachete diferite, tratate diferit de server și de rutere

Q10 · căutarea DNS, apoi memoria cache

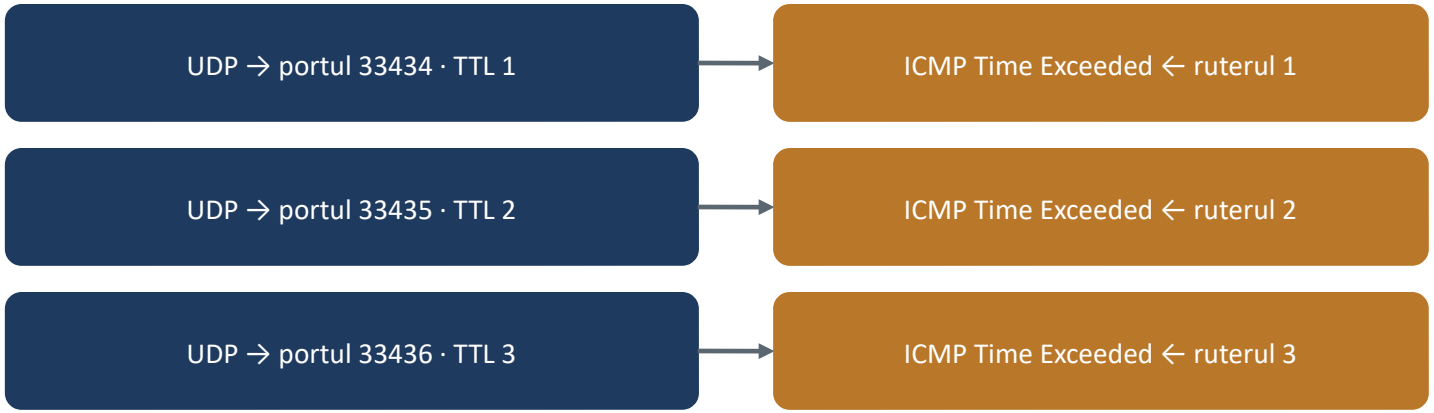


Primul curl: ratare (miss) → o interogare reală, cronometrați-o. Al doilea curl: **găsire** (hit) → niciun pachet DNS, până expiră TTL-ul înregistrării.

Ideea-cheie Stabilirea conexiunii este o măsurătoare gratuită a RTT în fiecare conexiune TCP.

TCP (Transmission Control Protocol) · DNS (Domain Name System) · RTT (round-trip time, timp dus-întors) · TTL (time to live, timp de viață) · SYN (synchronize, sincronizare) · ACK (acknowledgement, confirmare) · SEQ (sequence, secvență)

Sarcina 6 · Traceroute în captură



Q11 · protocolul, porturile destinație și TTL-ul primelor trei sonde

Q12 · în interiorul unui Time Exceeded

antet IP: ruter → voi

ICMP tipul 11, codul 0

copia sondei **voastre**: antetul ei IP + primii 8 octeți UDP (porturile)

De ce contează copia

Mai multe sonde sunt în zbor în același timp. Antetul și porturile citate îi spun lui traceroute **care** sondă a expirat și, deci, care salt a răspuns.

Ideea-cheie Fiecare răspuns poartă începutul sondei care l-a provocat — așa asociază traceroute răspunsurile cu întrebările.

ICMP (Internet Control Message Protocol) · UDP (User Datagram Protocol) · TTL (time to live, timp de viață) · IP (Internet Protocol)

Sarcina 7 · Captură din linia de comandă: tcpdump și tshark

captură pe ecran	`sudo tcpdump -i eth0 -n -c 20 'port 53 or port 80'`	→ 20 de pachete afișate, câte unul pe linie
captură în fișier	`sudo tcpdump -i eth0 -n -w cli.pcap 'host example.org' & apoi curl … ; sudo pkill tcpdump`	→ cli.pcap
citirea fișierului	`tshark -r cli.pcap`	→ o linie de rezumat pe pachet
extragerea câmpurilor	`tshark -r cli.pcap -Y http.request -T fields -e ip.src -e ip.dst -e http.host -e http.request.uri`	→ coloane: sursă, destinație, gazdă, cale

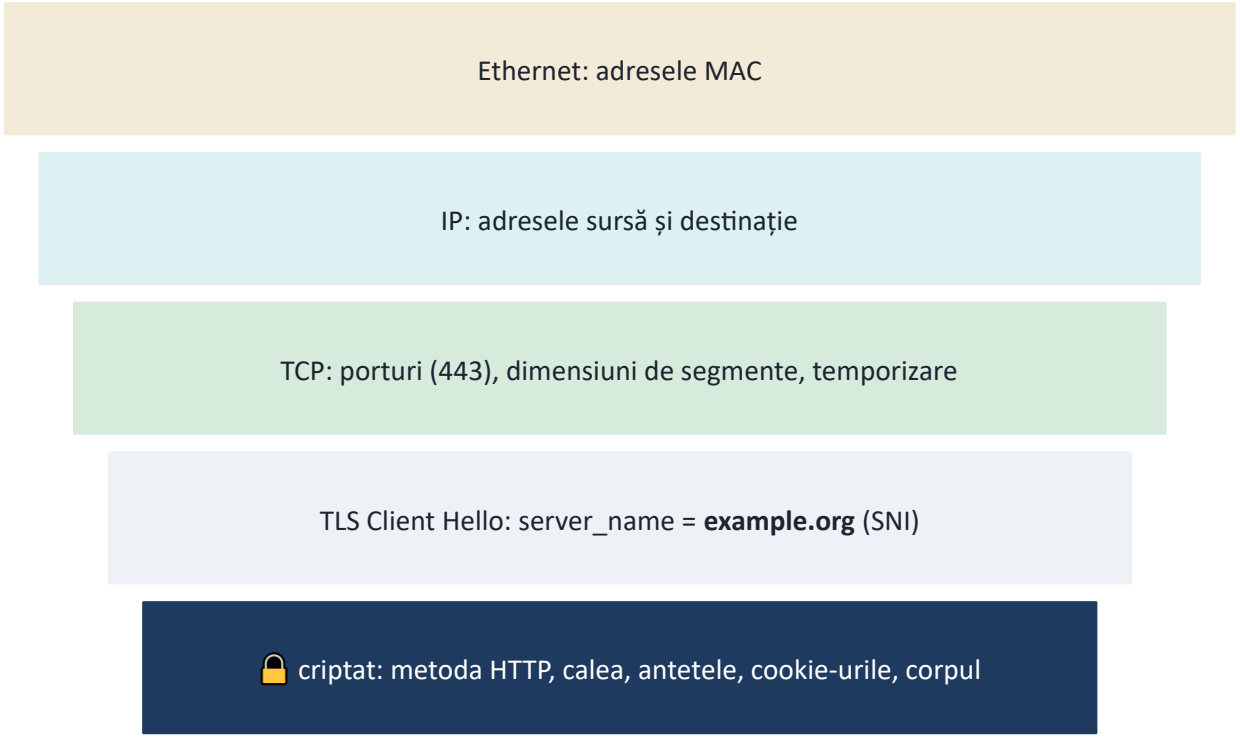
1 **-n**: numere, nu nume — fără căutări DNS inverse (care ar adăuga propriile pachete).

2 **Q13** · deschideți cli.pcap în Wireshark: este aceeași conversație ca în Sarcina 3?

Ideea-cheie tcpdump folosește sintaxa filtrelor de captură (BPF, Berkeley Packet Filter); tshark -Y folosește sintaxa filtrelor de afișare.

DNS (Domain Name System)

Sarcina 8 · HTTPS: ce rămâne vizibil



primele patru: **în clar** · ultimul: **criptat** de TLS (Transport Layer Security)

Q15 · un observator află totuși

- adresele voastre și ale serverului
- numele: SNI (Server Name Indication) și interogarea DNS de dinainte
- dimensiuni, momente, numărul de conexiuni

Q16 · criptat: nivelul aplicație

antetele de dedesubt circulă în clar; doar cele două capete au cheile → argumentul capăt-la-capăt

Ideea-cheie HTTPS ascunde ce cereți, nu de la cine cereți, când sau cât.

HTTPS (HyperText Transfer Protocol Secure) · HTTP (HyperText Transfer Protocol) · TCP (Transmission Control Protocol) · DNS (Domain Name System) · MAC (Media Access Control) · IP (Internet Protocol)

Dacă ceva nu funcționează (Linux)

Wireshark nu afișează nicio interfață	nu sunteți încă în grupul `wireshark`: rulați `groups`; deconectați-vă și reconectați-vă
nu apare nicio interogare DNS	răspunsul era în cache: `resolvectl flush-caches` sau interogați alt nume
niciun HTTP, doar TLS	redirecționare spre HTTPS: folosiți exact `http://example.org/`, `curl` fără `-L`
captura e inundată de alt trafic	folosiți un filtru de captură sau un filtru de afișare pe adresa serverului
`tcpdump: permission denied`	prefixați cu `sudo` sau folosiți permisiunea de grup a Wireshark

Ideea-cheie Majoritatea problemelor țin de permisiuni, de cache sau de redirecționări — verificați întâi aceste trei lucruri.

HTTPS (HyperText Transfer Protocol Secure) · HTTP (HyperText Transfer Protocol) · TLS (Transport Layer Security) · DNS (Domain Name System)

Partea D · Laboratorul 2 pe Windows

A · Cursul 2
Arhitectură și performanță

→

B · Seminarul 2
Antete, cozi, debit

→

C · Laboratorul 2 · Linux
Wireshark și tcpdump

→

D · Laboratorul 2 · Windows
Wireshark și pktmon

În această parte

Înainte de a începe: cum capturează Windows

1–2 · Prima captură, filtre

3 · O cerere web, nivel cu nivel

4–5 · Perspective asupra conversației și RTT

6 · tracert la microscop

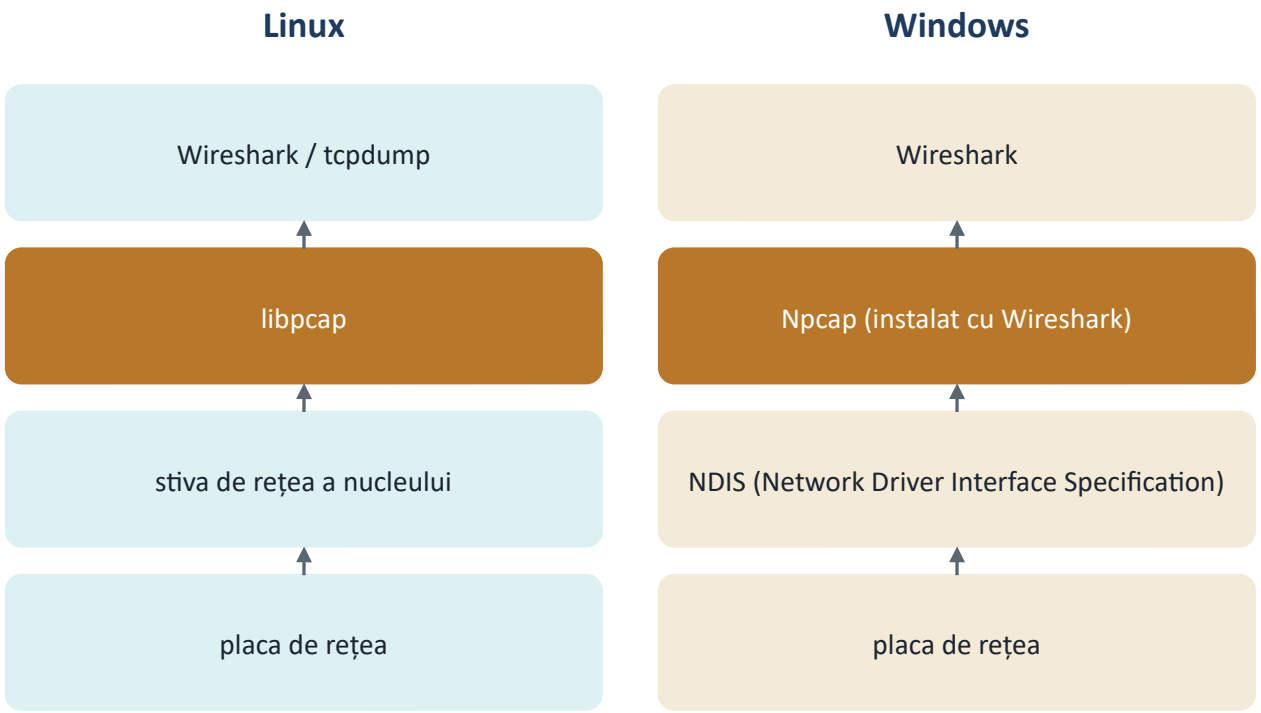
7 · pktmon și tshark

8 · HTTPS: ce rămâne vizibil

Linux și Windows alăturate

HTTPS (HyperText Transfer Protocol Secure) · RTT (round-trip time, timp dus-întors)

Înainte de a începe (Windows): cum ajung pachetele la Wireshark



Npcap = driverul de captură de pachete al proiectului Nmap · libpcap = biblioteca de captură de pachete

Instalare
Wireshark de pe wireshark.org; acceptați **Npcap**. Instalarea cere drepturi de administrator; captura de după, nu.

Terminal
PowerShell; comenzile marcate *(admin)* cer „Run as administrator”.

Notă Wi-Fi
Windows îi dă Wireshark cadre care **arată ca** Ethernet; antete 802.11 reale doar în modul monitor (la fel pe Linux).

De predat
răspunsurile la **Q1–Q16** + **web.pcapng** din Sarcina 3; valorile afișate sunt ilustrații.

Ideea-cheie Același Wireshark, altă instalație: Npcap joacă pe Windows rolul pe care libpcap îl joacă pe Linux.

Laboratorul 2 (Windows) · cele două ore



1 O primă captură și un tur al Wireshark

2 Filtre de captură și filtre de afișare

3 O cerere web, nivel cu nivel

4 Întreaga conversație: fluxuri, statistici, graful fluxului

5 Măsurarea RTT (round-trip time, timp dus-întors) din stabilirea conexiunii

6 tracert la microscop

7 Captură din linia de comandă: pktmon și tshark

8 HTTPS: ce rămâne vizibil

Ideea-cheie Structură identică cu fișa Linux, ca răspunsurile să poată fi comparate sarcină cu sarcină.

HTTPS (HyperText Transfer Protocol Secure)

Sarcinile 1–2 · Prima captură pe Windows

Ethernet



Wi-Fi



adaptor VPN

vEthernet (Hyper-V)



← dublu clic pe cel care se mișcă

VPN = virtual private network, rețea privată virtuală

```
curl.exe -s http://example.org/ -o NUL
```

Q1 · alte protocoale pe care le veți vedea fără să le cereți:

ARP
Address Resolution Protocol

mDNS
multicast DNS (Domain Name System)

SSDP
Simple Service Discovery Protocol

LLMNR
Link-Local Multicast Name Resolution

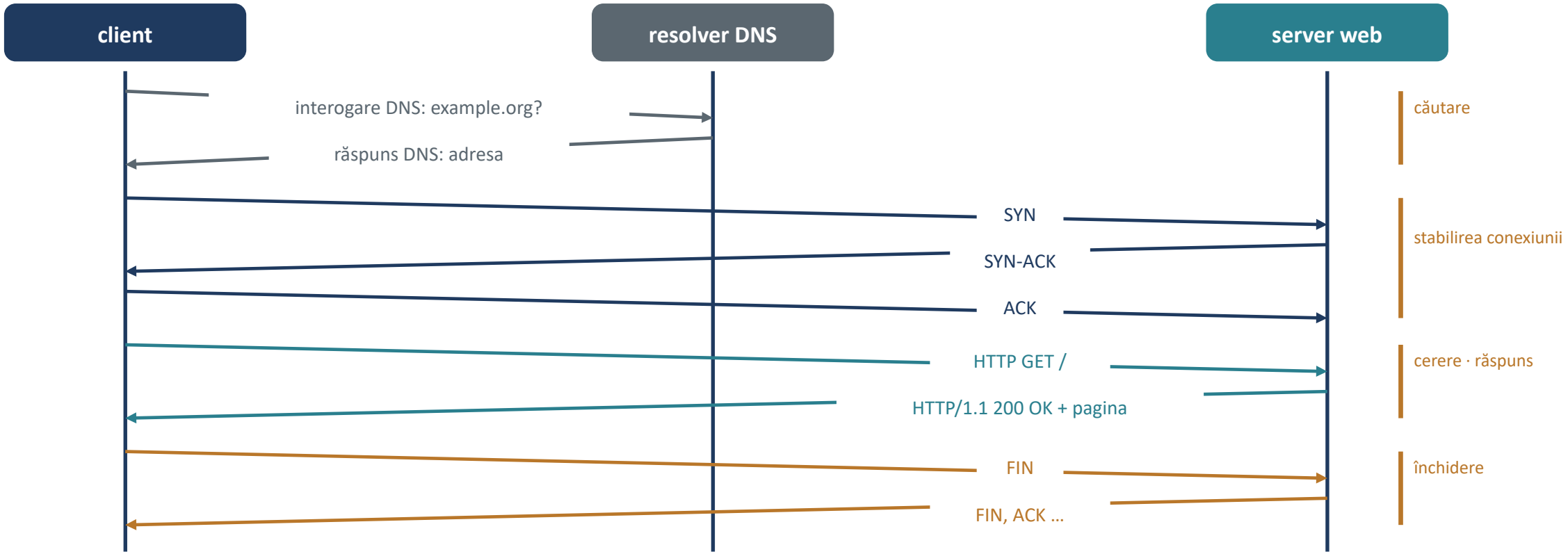
Q2–Q3 · asociați liniile de detaliu celor cinci niveluri · filtrele sunt exact cele din fișa Linux:
captură `port 53 or port 80` · afișare `dns`, `http`, `tcp.port == 80`, `tcp.flags.syn == 1` · propria adresă: `ipconfig`

Ideea-cheie Pe Windows alegeți adaptorul după trafic, nu după nume: adaptoarele virtuale nu arată nimic.

mDNS (multicast DNS, Domain Name System)

Sarcina 3 · O cerere web pe Windows

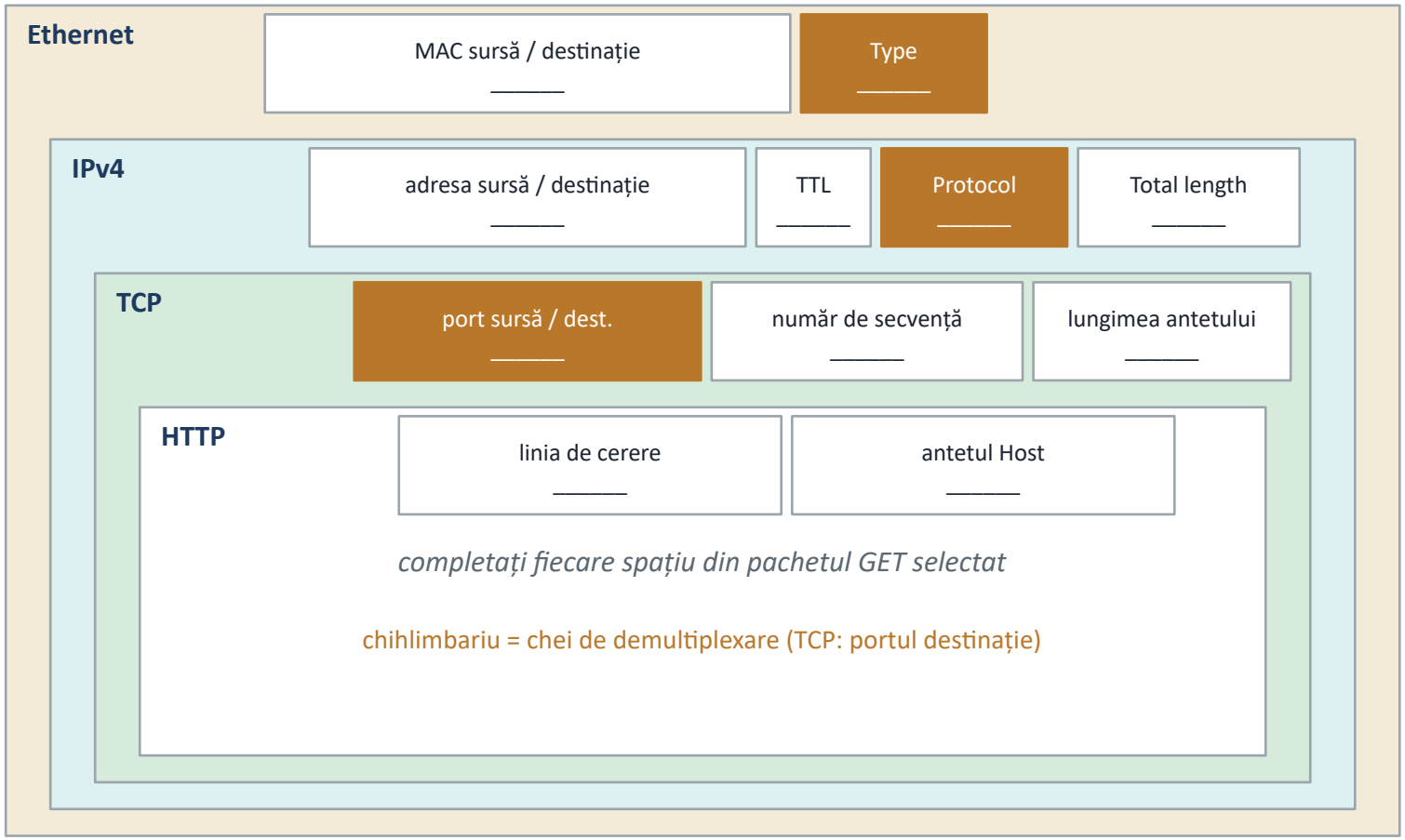
```
ipconfig /flushdns   →   captură   →   curl.exe -s http://example.org/ -o NUL   →   stop   →   File → Save As… web.pcapng
```



Ideea-cheie Pachetele sunt aceleași ca pe Linux — diferă doar comenzile care le produc.

HTTP (HyperText Transfer Protocol) · DNS (Domain Name System) · SYN (synchronize, sincronizare) · ACK (acknowledgement, confirmare) · FIN (finish, încheiere)

Sarcina 3 · Pachetul GET și Q5–Q7 pe Windows



1 **Q5** · comparați MAC-ul destinație al GET-ului cu al porții implicite: `arp -a` sau `Get-NetNeighbor` — coincid.

2 **Q6** · TTL-ul vostru la plecare: **128** (Windows). TTL-ul răspunsului 52 → din 64 → 12 rutere. **Q7** · 14 + 20 + 20 = 54 de octeți de antete înainte de HTTP.

Ideea-cheie Aceleași chei de demultiplexare, aceleași antete; Windows diferă doar prin TTL-ul inițial și prin instrumente.

HTTP (HyperText Transfer Protocol) · IPv4 (Internet Protocol version 4) · TCP (Transmission Control Protocol) · MAC (Media Access Control) · TTL (time to live, timp de viață)

Sarcinile 4–5 · Perspective asupra conversației și RTT pe Windows

Follow → TCP Stream

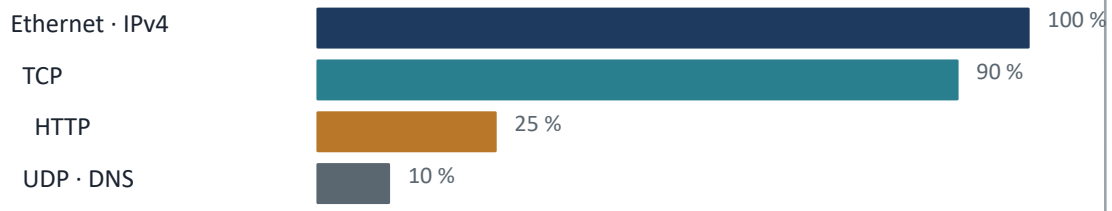
datele de aplicație din ambele sensuri, colorate

```
GET / HTTP/1.1  ·  Host: example.org
```

```
HTTP/1.1 200 OK  ·  Content-Type: text/html
```

Statistics → Protocol Hierarchy

cum se împart octeții între protocoale



Statistics → Conversations

fiecare pereche de capete, octeții schimbați

adresa A ↔ adresa B	pachete	octeți
10.0.2.15 ↔ resolver	2	≈ 160
10.0.2.15 ↔ server	≈ 10	≈ 2 300

Statistics → Flow Graph

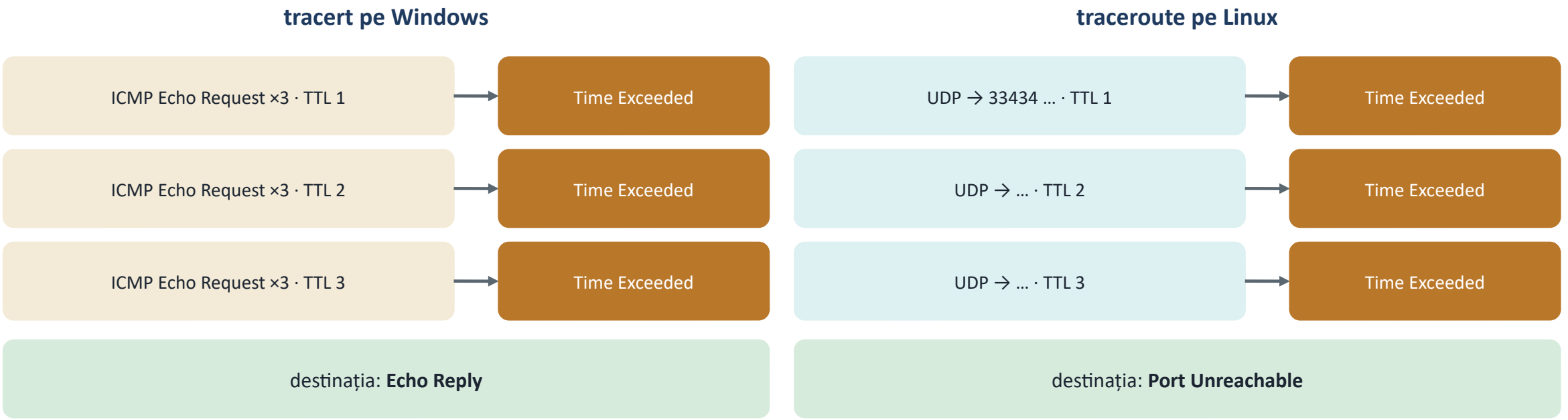
schimbul ca diagramă timp–secvență



Ideea-cheie Q9–Q10 · ping -n 5 înlocuiește ping -c 5; ipconfig /displaydns arată numele din cache și TTL-ul rămas.

HTTP (HyperText Transfer Protocol) · IPv4 (Internet Protocol version 4) · TCP (Transmission Control Protocol) · UDP (User Datagram Protocol) · DNS (Domain Name System) · RTT (round-trip time, timp dus-întors) · TTL (time to live, timp de viață) · SYN (synchronize, sincronizare) · ACK (acknowledgement, confirmare)

Sarcina 6 · tracert trimite ICMP, nu UDP



1 **Q11–Q12** · tipul sondei, TTL salt după salt, sonde pe salt; răspunsurile sunt ICMP Time Exceeded (tipul 11) și citează începutul Echo Request-ului **vostru** — așa știe tracert care sondă a expirat.

Ideea-cheie Același truc cu TTL, altă sondă: ICMP Echo pe Windows, UDP spre porturi mari pe Linux.

ICMP (Internet Control Message Protocol) · UDP (User Datagram Protocol) · TTL (time to live, timp de viață)

Sarcina 7 · pktmon: monitorul de pachete încorporat în Windows

(admin) pornire

```
pktmon start --capture --comp nics --pkt-size 0 -f cli.etl
```

trafic

```
curl.exe -s http://example.org/ -o NUL
```

oprire

```
pktmon stop
```

conversie

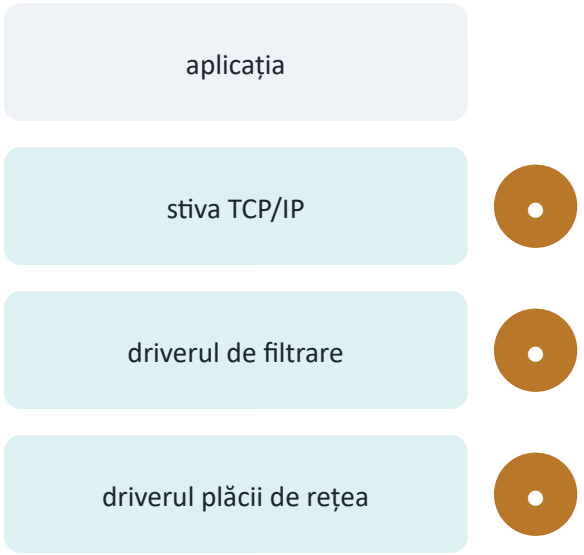
```
pktmon etl2pcap cli.etl -o cli.pcapng
```

analiză

```
tshark.exe -r cli.pcapng -Y http.request -T fields -e ip.src -e ip.dst -e http.host -e
http.request.uri
```

ETL = Event Trace Log · tshark.exe se află în C:\Program Files\Wireshark

Q13 · de ce duplicate?

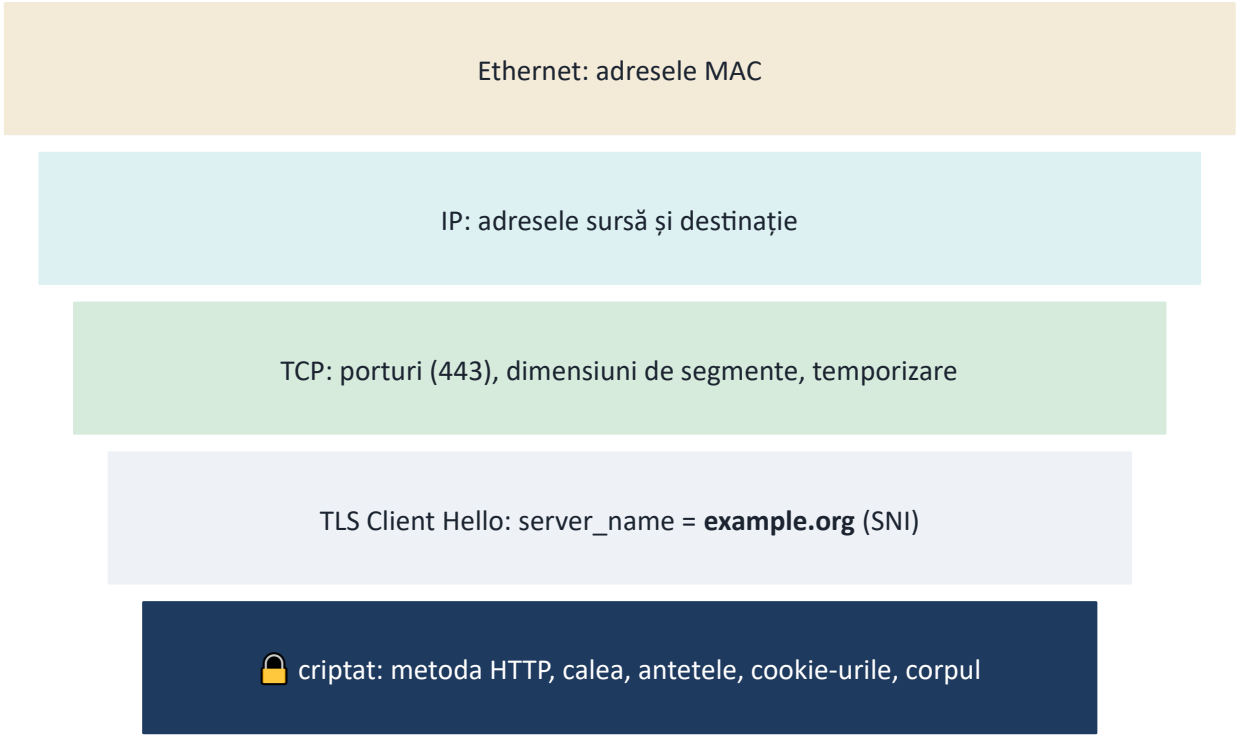


pktmon poate înregistra un pachet în **mai multe puncte** ale stivei, așa că acesta poate apărea de mai multe ori.

Ideea-cheie pktmon → .etl → etl2pcap → .pcapng: calea încorporată de captură când Wireshark nu este instalat.

TCP (Transmission Control Protocol) · IP (Internet Protocol)

Sarcina 8 · HTTPS pe Windows: ce rămâne vizibil



primele patru: **în clar** · ultimul: **criptat** de TLS (Transport Layer Security)

Q15 · un observator află totuși

- adresele voastre și ale serverului
- numele: SNI (Server Name Indication) și interogarea DNS de dinainte
- dimensiuni, momente, numărul de conexiuni

Q16 · criptat: nivelul aplicație

antetele de dedesubt circulă în clar; doar cele două capete au cheile → argumentul capăt-la-capăt

Ideea-cheie Criptarea protejează conținutul capăt-la-capăt; adresele, numele, dimensiunile și momentele rămân vizibile pe orice sistem.

HTTPS (HyperText Transfer Protocol Secure) · HTTP (HyperText Transfer Protocol) · TCP (Transmission Control Protocol) · DNS (Domain Name System) · MAC (Media Access Control) · IP (Internet Protocol)

Linux și Windows alăturate

Scop	Linux	Windows
driverul de captură	nucleul (libpcap)	Npcap
captură grafică	Wireshark	Wireshark
captură încorporată din linia de comandă	tcpdump	pktmon (admin)
analiză din linia de comandă	tshark	tshark.exe
golirea memoriei cache DNS	resolvectl flush-caches	ipconfig /flushdns
sondele traceroute	UDP spre porturi mari (implicit)	ICMP Echo Request
TTL inițial tipic	64	128

Ideea-cheie Învățați scopul o singură dată; pe fiecare sistem se schimbă doar numele instrumentului și câteva valori implicite.

Npcap (driverul de captură de pachete al proiectului Nmap) · ICMP (Internet Control Message Protocol) · UDP (User Datagram Protocol) · DNS (Domain Name System) · TTL (time to live, timp de viață)

Dacă ceva nu funcționează (Windows)

Wireshark nu afișează nicio interfață	→ lipsește Npcap: rulați din nou programul de instalare Wireshark și selectați Npcap
adaptorul ales nu arată nimic	→ un adaptor virtual (VPN, virtual private network, rețea privată virtuală; Hyper-V): alegeți-l pe cel a cărui linie de activitate se mișcă
nu apare nicio interogare DNS	→ numele era în cache: `ipconfig /flushdns` din nou, chiar înainte de cerere
`pktmon` raportează „access denied”	→ deschideți terminalul cu „Run as administrator”
`curl` afișează un obiect, nu o pagină	→ acela era aliasul din PowerShell: tastați `curl.exe`

Ideea-cheie Pe Windows, vinovații obișnuiți sunt driverul, adaptorul și aliasul curl.

Npcap (driverul de captură de pachete al proiectului Nmap) · DNS (Domain Name System)

Săptămâna viitoare

Cursul 3

arhitecturi de aplicații · socket-uri și serviciile de care au nevoie aplicațiile · Web-ul și HTTP/1.1

Seminarul 3

probleme despre timpi de încărcare a paginilor, mesaje și încadrare

Laboratorul 3

un client și un server TCP cu socket-uri — în C pe Linux, cu Winsock pe Windows

Ideea-cheie Săptămâna aceasta ați citit pachete trimise de programele altora; săptămâna viitoare scrieți programele.

HTTP (HyperText Transfer Protocol) · TCP (Transmission Control Protocol)